



University of
Nottingham

UK | CHINA | MALAYSIA

A large, high-resolution image of the Earth as seen from space, showing the Western Hemisphere with North and South America. The image is framed by a thin white border.

COMP-2024 / COMP-2039
**Artificial Intelligence
Methods**

Lecture 8
Hyper-heuristics II



Learning Outcomes

IDENTIFY

1. Designing Metaheuristics for Examination Timetabling
2. Graph-based Hyper-heuristic
3. Hyper-heuristic Framework
4. Generation Hyper-heuristics
5. Genetic Programming



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



University of
Nottingham

UK | CHINA | MALAYSIA

Designing Metaheuristics for Examination Timetabling



Examination timetabling

- A number of exams ($e_1, e_2, e_3, \dots, e_E$), taken by different students ($s_1, s_2, s_3, \dots, s_S$), need to be scheduled to a limited time periods ($t_1, t_2, t_3, \dots, t_T$) and certain rooms ($r_1, r_2, r_3, \dots, r_R$).
- **Hard Constraints**
 - Exams taken by common students can't be assigned to the same time period.
 - Room capacity can't be exceeded.
 - Cannot be violated
- **Soft Constraints**
 - Separation between exams.
 - Large exams scheduled early.
 - Could be violated sometimes

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Designing a **Local Search Metaheuristic** for Examination Timetabling

- **Representation:** An array of pair of integers, one representing the period assignment and the other representing the room assignment. The array size is the number of events E and each period entry has a value from 1 to T , while room has a value from 1 to R (integer encoding).
- **Initialisation:** randomly assign a period and a room (integer values within the given range) for each event.
- **Objective function:** Number of constraint violations.



Designing **Iterated Local Search** for Examination Timetabling

- **Neighbourhood (perturbation) operator:**
 - OP1: Randomly pick an event and reschedule to random period.
 - OP2: Randomly pick an event and assign a different room.
 - OP3: Randomly pick an event and reschedule to random period, also assign a different room.
 - Any of the above can be parametrised, e.g. pick X number of events and apply an operator.
 - More elaborate operators can be designed.
- **Local Search:** RMHC, DBHC, SDHC, NDHC, in which the neighbourhood operator is OP3.
- **Acceptance Criterion:** accept improving and equal moves (IE) (non-worsening): accept a new solution s' if and only if $f(s') \leq f(s^*)$.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Designing **Simulated Annealing** for Examination Timetabling

- **Initial Temperature:** Objective value of the initial solution generated.
- **Cooling Schedule:** Geometric cooling, $\alpha=0.99$.
- **Termination criteria:** Stop and return the best solution found so far when the total number of violation is 0 or maximum number of iterations is exceeded.



University of
Nottingham

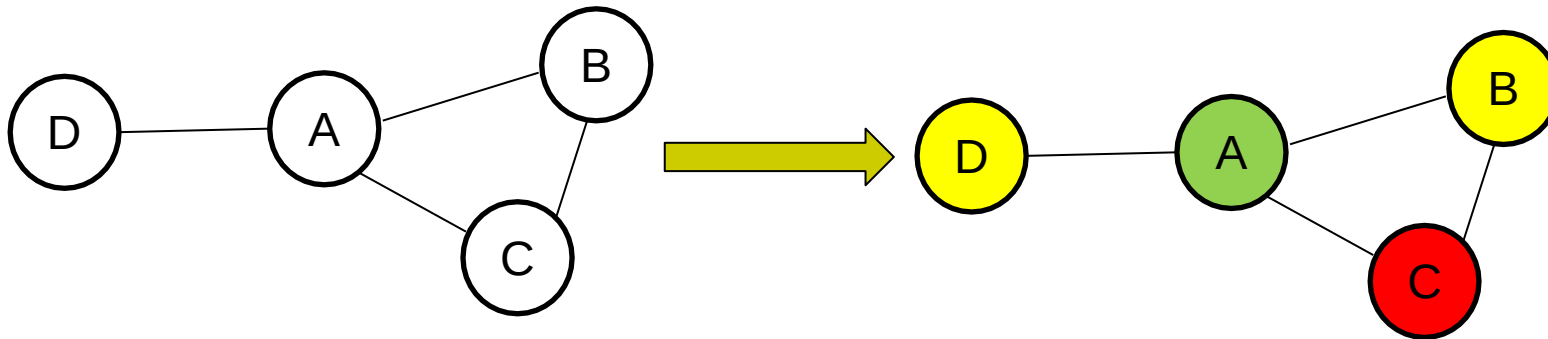
UK | CHINA | MALAYSIA

A Graph-based Hyper-heuristic



Graph Colouring

- An assignment of labels traditionally called "colours" to elements of a **graph** subject to certain *constraints*.
- A way of colouring the vertices of a graph such that *no two adjacent vertices share the same colour*;
- This is called a **vertex colouring**.





Graph Colouring Problem

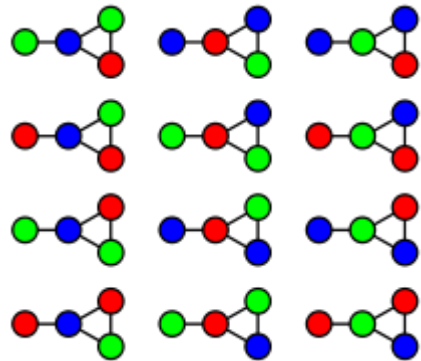
- The **graph coloring problem** entails assigning colours to vertices of a graph while adhering to particular limits and *constraints* → **no two neighbouring vertices, adjacent edges have the same colour.**
- This is known as the *chromatic number*, and the graph is known as a *properly coloured graph*.
 - The *chromatic number* of a graph, denoted as $\chi(\mathbf{G})$, is the **minimum number of colors** required to color the vertices of the graph such that no two adjacent vertices share the same color.
- Colours, order of coloring, colour assignment, and other constraints are established on the graph while graph coloring. As a result, vertices or areas with the same colour create *independent sets*.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Graph Colouring

- **k -colouring problem:** Can the vertices of a graph be coloured using k colours so that no two vertices connected by an edge have the same colour?



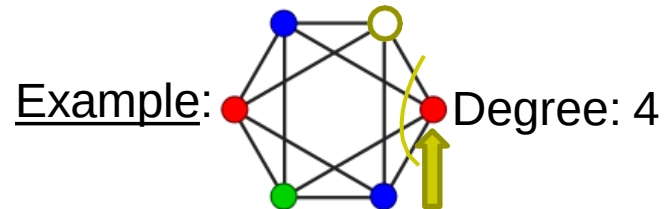
This graph can be 3-colored in 12 different ways.

- **Minimum colouring problem** is an *NP-hard problem*: colour the vertices of a graph using **optimal (minimum) number of colours**, so that no two vertices connected by an edge have the same colour.

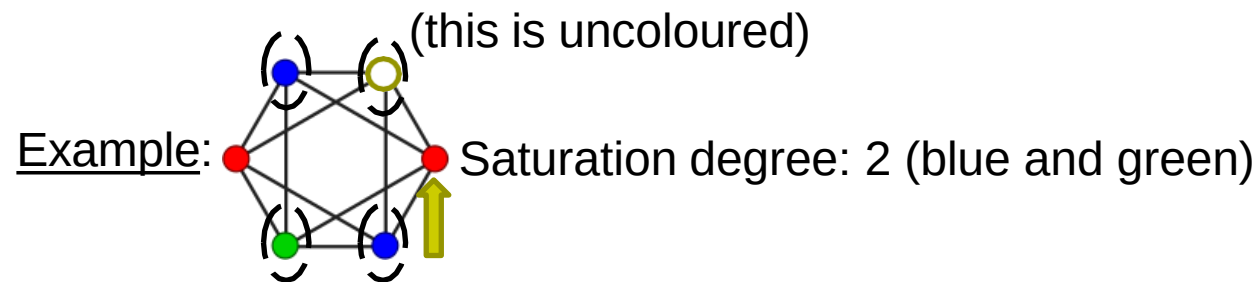


Degree and Saturation Degree of a Vertex

- **Degree of a vertex:** number of edges connected to that vertex.



- **Saturation degree of a vertex:** number of *differently coloured* vertices already connected to it.



- Can we use *saturation degree* in a heuristic to construct a solution to a graph colouring problem?



Heuristic Algorithms

■ Greedy Coloring

- Assigns colors sequentially, choosing the *lowest available degree (number of connections) for each vertex*.
- Fast but does not guarantee the optimal solution.
- Example: The Welsh-Powell algorithm sorts vertices by degree and colors them greedily. <https://www.geeksforgeeks.org/welsh-powell-graph-colouring-algorithm/>

■ Largest First and Smallest Last Heuristics

- Order vertices by highest or lowest degree and then apply greedy coloring.



Graph Colouring Heuristics

- **Largest Degree Heuristic (LD)**
- **Saturation Degree Heuristic (SD)**
- Largest Weighted Degree Heuristic (LWD)
- Color Degree Heuristic (CD)
- Random Ordering Heuristic (RO)
- Largest Enrolment Heuristic (LE)
- Degree of Saturation Heuristic (DS)

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Largest Degree Heuristic (LD)

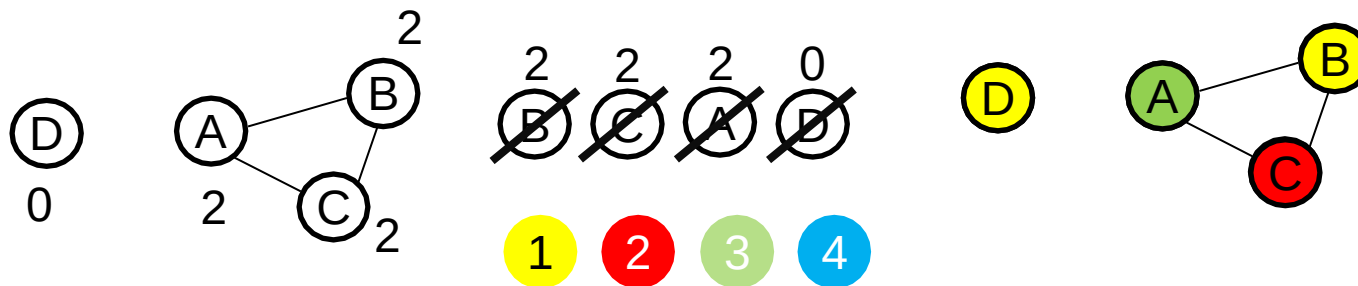
- Select the vertex with the *highest degree (most edges)* (static feature) and color it *first*.
- **Procedure:**
 1. Calculate the degree of all vertices.
 2. Sort vertices in decreasing order of degree.
 3. Color the first vertex in the list and repeat.



Graph Colouring Heuristics

■ Largest Degree:

1. Compute the degree of all vertices.
2. Sort the vertices from **largest degree** to smallest.
3. Colour the first vertex in the list with the next colour (starting with the first) that is different than its neighbours.
4. Delete the vertex from the list go to the previous step unless no vertices left.





Saturation Degree Heuristic (SD)

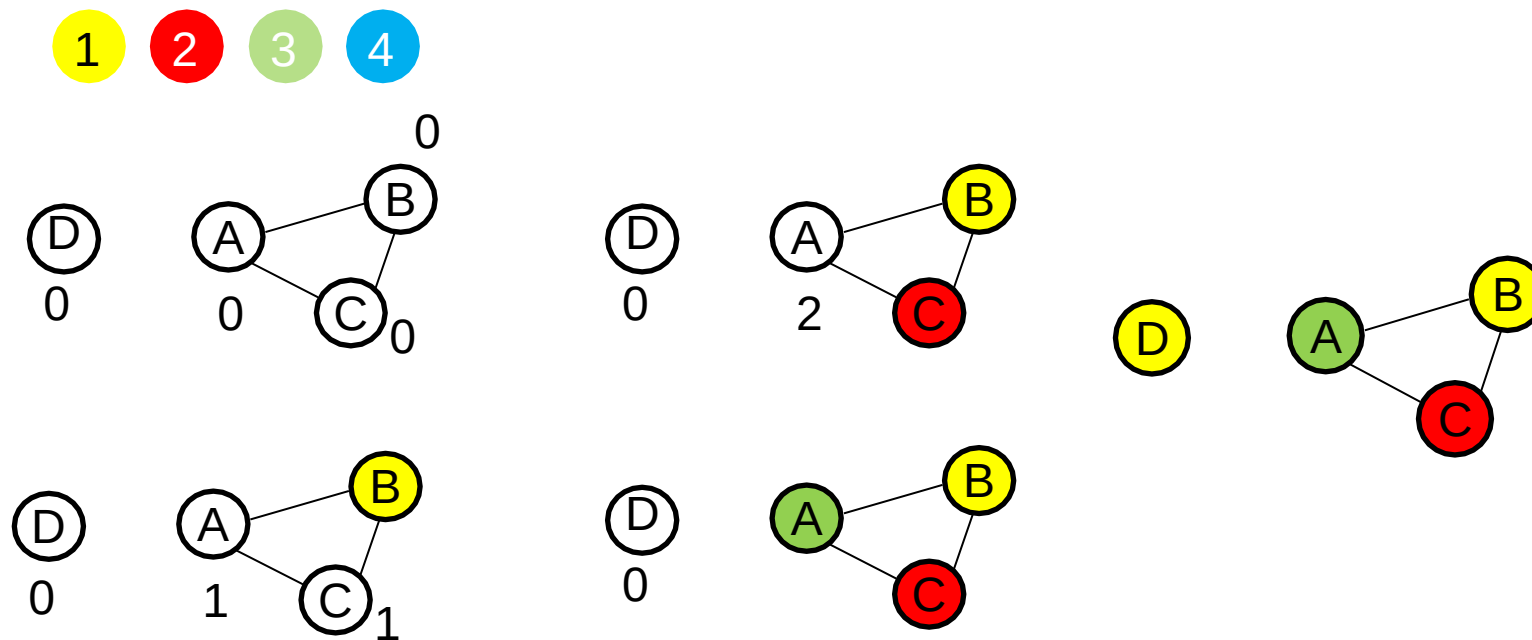
- Choose the vertex with the *highest saturation degree*, which is the *number of different colors* (dynamic feature) assigned to its neighbours.
- **Procedure:**
 1. For each uncolored vertex, calculate its saturation degree.
 2. Color the vertex with the highest saturation degree.
 3. Update saturation degrees and repeat until all vertices are colored.



Graph Colouring Heuristics

■ Saturation Degree:

- Use saturation degree at each step in the previous approach.



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Stopping Criteria

- Fixed Number of Iterations
- No More Conflicts (Valid Coloring Found)
- No Improvement Over Consecutive Iterations
- Converging to a Stable Coloring
- Time Limit
- Maximum Number of Colors Reached
- Satisfactory Solution Found (Soft Constraints) - when a "good enough" solution is found.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Example 1

Example Graph:

We will use a simple undirected graph with vertices $V = \{A, B, C, D, E\}$ and edges:

$$E = \{(A, B), (A, C), (B, C), (B, D), (C, E), (D, E)\}.$$



Example 1: Largest Degree Heuristic (LD)

The Largest Degree Heuristic selects the vertex with the highest degree (most connections) to color first. Then it proceeds to the next highest degree vertex, and so on.

Steps:

1. Compute the degree of each vertex:
 - $\deg(A) = 2$
 - $\deg(B) = 3$
 - $\deg(C) = 3$
 - $\deg(D) = 2$
 - $\deg(E) = 2$
2. Order vertices by descending degree: B, C, A, D, E .
3. Color vertices sequentially, ensuring no two adjacent vertices have the same color.

Coloring Process:

- B : Color 1.
- C : Adjacent to B , so use Color 2.
- A : Adjacent to B , so use Color 2.
- D : Adjacent to B , so use Color 2.
- E : Adjacent to C and D , so use Color 1.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & D



Example 1: Saturation Degree Heuristic (SD)

The **Saturation Degree Heuristic** selects the vertex with the highest saturation degree (number of distinct colors used by its neighbors). If there is a tie, choose the vertex with the largest degree.

Steps:

1. Start with no vertices colored. Initialize saturation degrees for all vertices to 0.
2. At each step, select the vertex with the highest saturation degree. If tied, use the largest degree.

Coloring Process:

- **Step 1:** Start with B (highest degree). Color B with Color 1.
- **Step 2:** C (saturation = 1). Color C with Color 2.
- **Step 3:** A (saturation = 1). Color A with Color 2.
- **Step 4:** D (saturation = 1). Color D with Color 2.
- **Step 5:** E (saturation = 2, adjacent to C and D). Color E with Color 1.

Result:

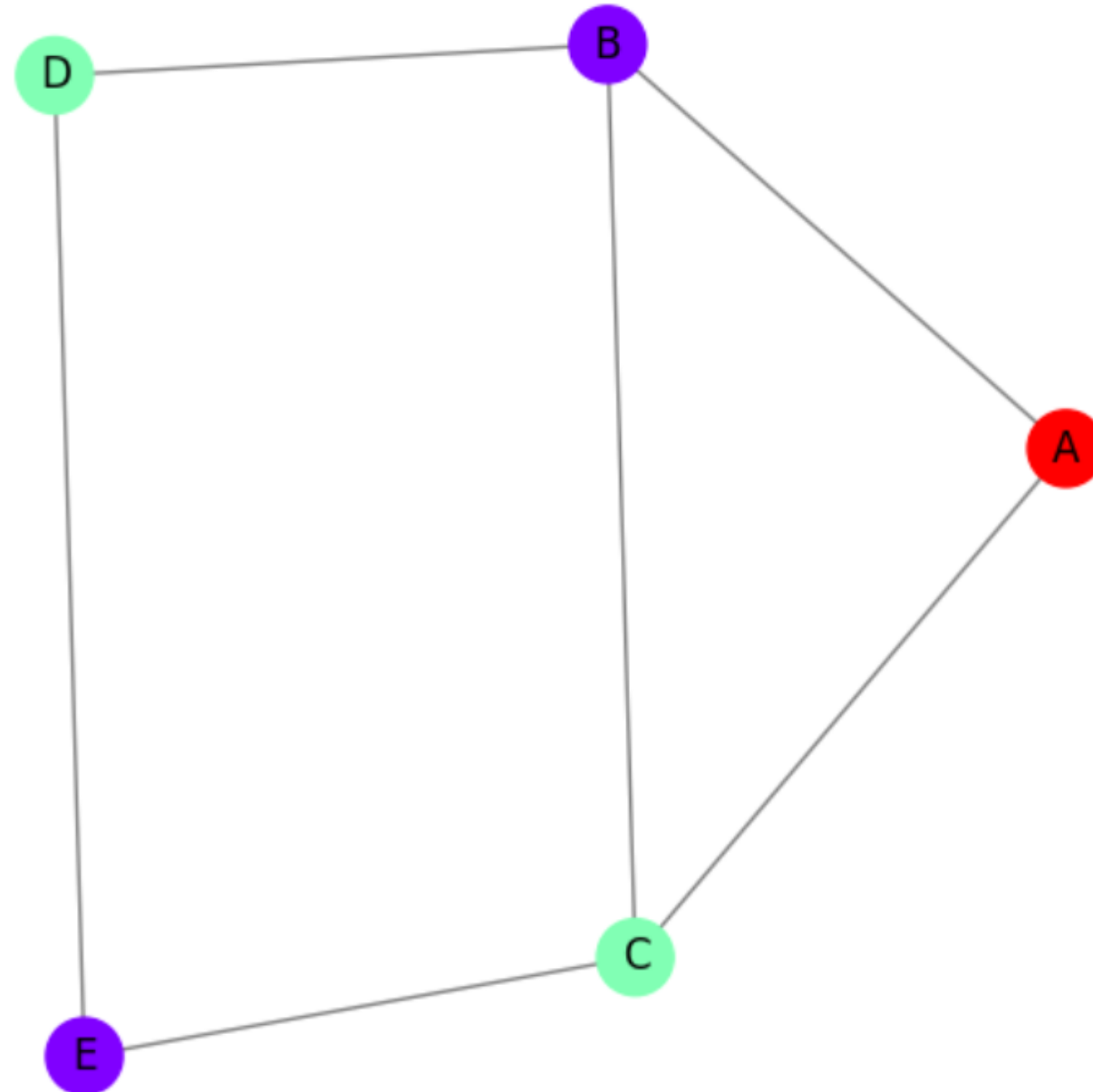
Vertex Colors: $A(2), B(1), C(2), D(2), E(1)$



Example 1

Final Color Assignments: {'B': 1, 'C': 2, 'A': 3, 'D': 2, 'E': 1}

Final Graph Coloring



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Example

- **LD** prioritizes *degree* (number of edges – static feature) initially, leading to a *static ordering*.
- **SD** dynamically prioritizes based on *saturation* (number of coloured vertices – dynamic feature), *adapting* as the coloring progresses.

Heuristic	Selection Strategy	Coloring Strategy
Largest Degree First	Picks nodes with the highest number of edges	Colors nodes in descending degree order
Saturation Degree	Picks nodes with the most distinct neighbor colors	Colors nodes dynamically based on saturation

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



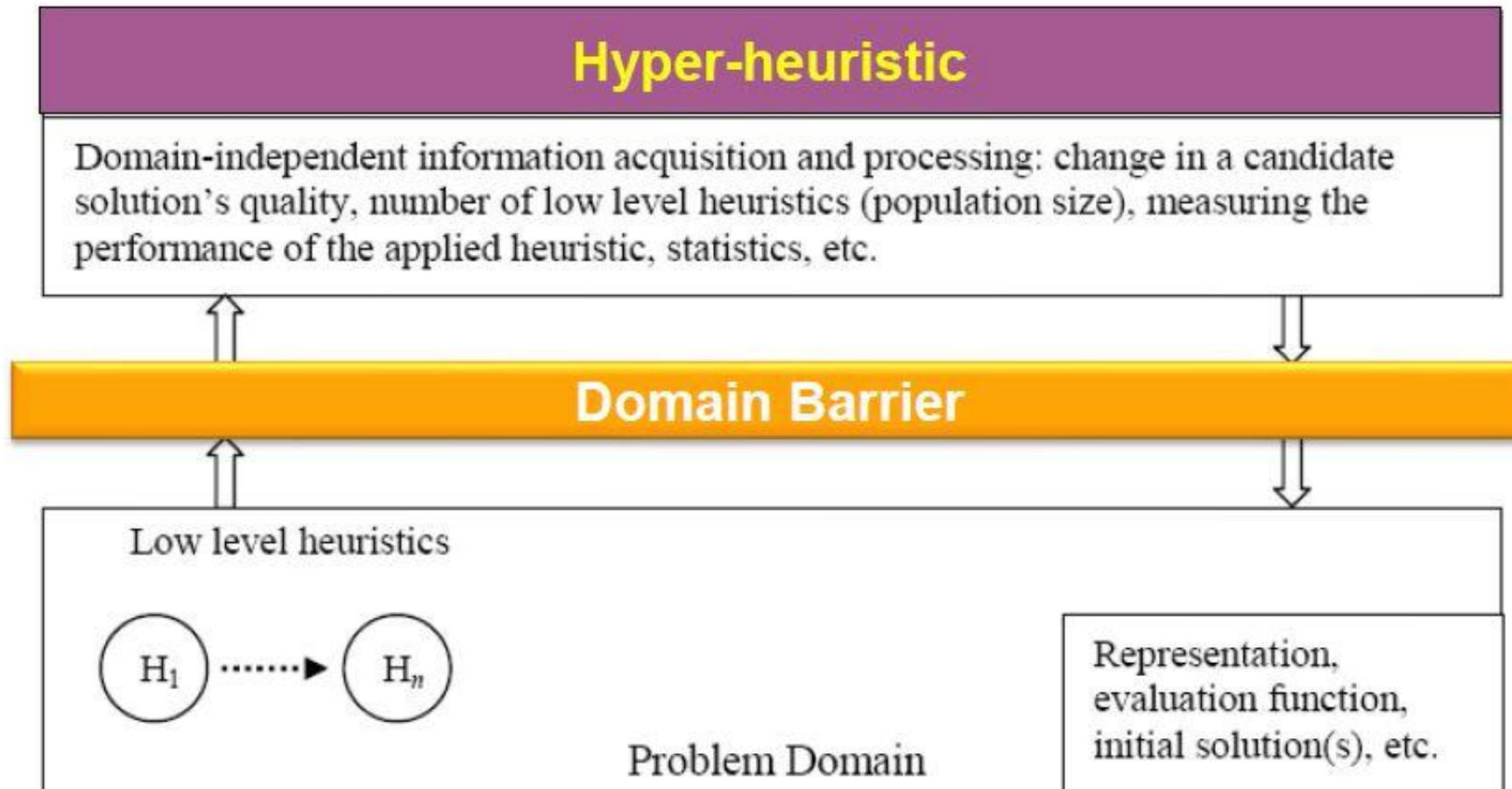
University of
Nottingham

UK | CHINA | MALAYSIA

Designing Graph- based Hyper- Heuristics for Examination Timetabling



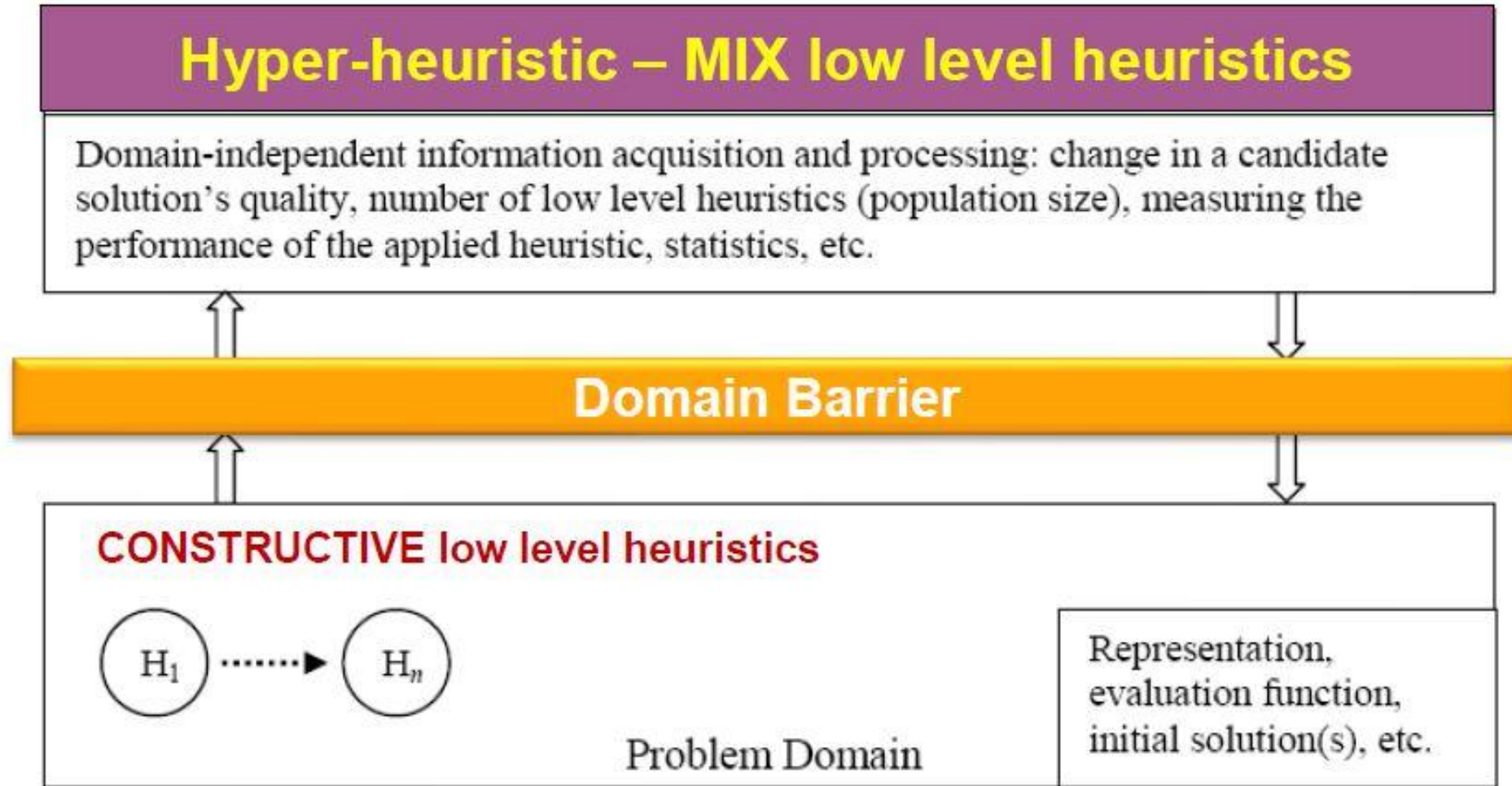
A Hyper-heuristic Framework



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Methodologies to select constructive heuristics



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Graph-based hyper-heuristics

- A *general framework Hyper Heuristic (GHH)* - employing a set of low-level constructive graph colouring heuristics.
- **Low level heuristics**: sequential methods that order events by the difficulties of assigning them.
 - 5 graph colouring heuristics
 - Random ordering strategy
- Applied to exam and course timetabling problem.

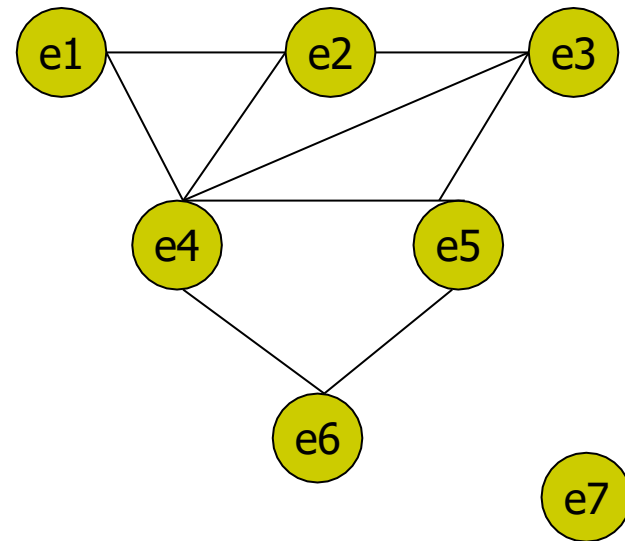
ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Examination timetabling

■ How can we represent/model this problem?

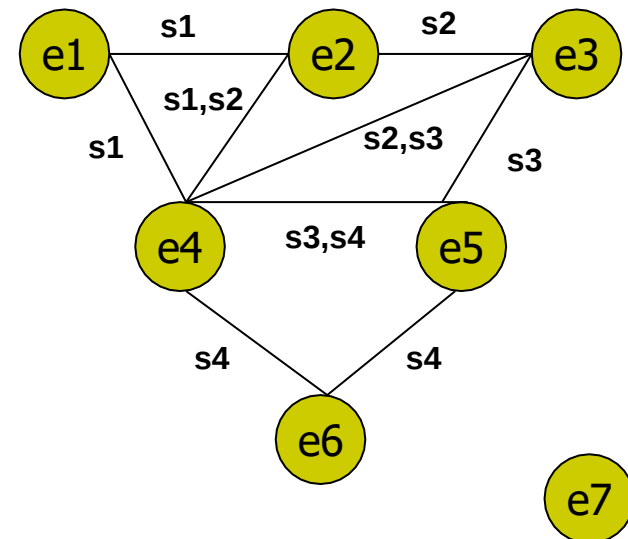
- There are 7 exams, $e1 \sim e7$.
- 5 students taking different exams.
 - $s1$: $e1, e2, e4$
 - $s2$: $e2, e3, e4$
 - $s3$: $e3, e4, e5$
 - $s4$: $e4, e5, e6$
 - $s5$: $e7$
- let's ignore room allocation.





Examination timetabling

- Can be modelled as a **graph colouring problem**.
 - **Nodes**: exams.
 - **Edges**: adjacent exams (nodes) have *common* students.
 - **Colours**: time periods.
 - **Objective**: assign colours (time periods) to nodes (exams), adjacent nodes with different colour, minimising time periods used.



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Graph-based hyper-heuristics

Graph Coloring Heuristics	Ordering strategies
Largest degree (LD)	Number of clashed events
Largest weighted degree (LW)	LD with number of common students
Largest enrolment (LE)	Number of students
Saturation degree (SD)	Number of valid remaining time periods
Colour degree (CD)	Number of clashed events that are already scheduled
+	
Random ordering (RO)	Orders a number events randomly

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Pseudo-code of **Tabu Search** graph based hyper-heuristic

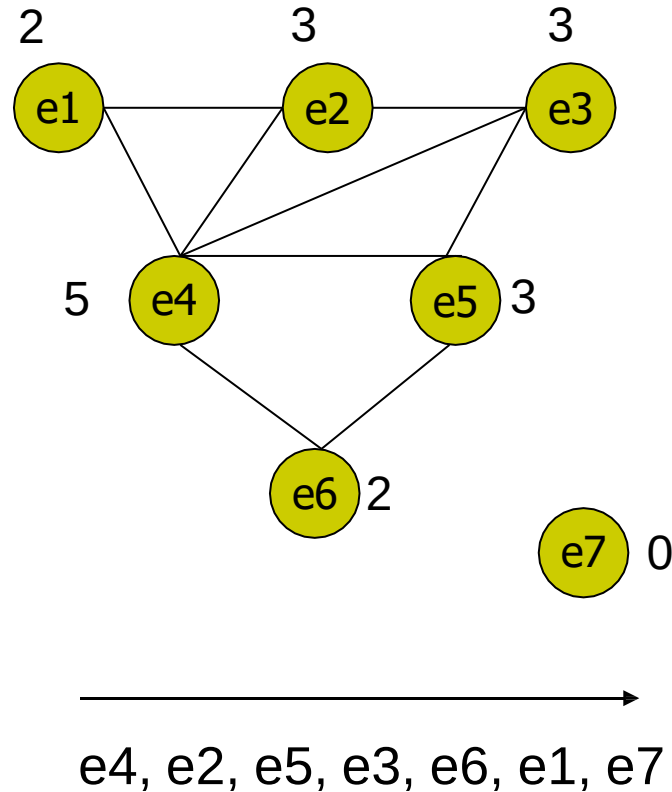
```
initial heuristic list  $h_l = \{h_1 \ h_2 \ h_3 \ ... \ h_k\}$ 
//Begin of Tabu Search
for  $i = 0$  to  $i = (5 * \text{the number of events})$  //number of iterations
     $h = \text{change two heuristics in } h_l$  //a move in Tabu Search
    if  $h$  does not match a heuristic list in 'failed list'
        if  $h$  is not in the tabu list //h is not recently visited
            for  $j =$  to  $j = k$  //h is used to construct a complete solution
                schedule the first events in the event list ordered using  $h_j$ 
            if no feasible solution can be obtained
                store  $h$  into the 'failed list' //update "failed list"
            else if cost of solution  $c < \text{the best cost } c_g \text{ obtained}$ 
                save the best solution,  $c_g = c$  //keep the best solution
                add  $h$  into the tabu list
                remove the first item from the tabu list if its length  $> 9$ 
                 $h_l = h$ 
        //end if
    Deepest descent on the complete solution obtained
//end of Tabu Search
output the best solution with cost of  $c_g$ 
```

number of events = $N * k$

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



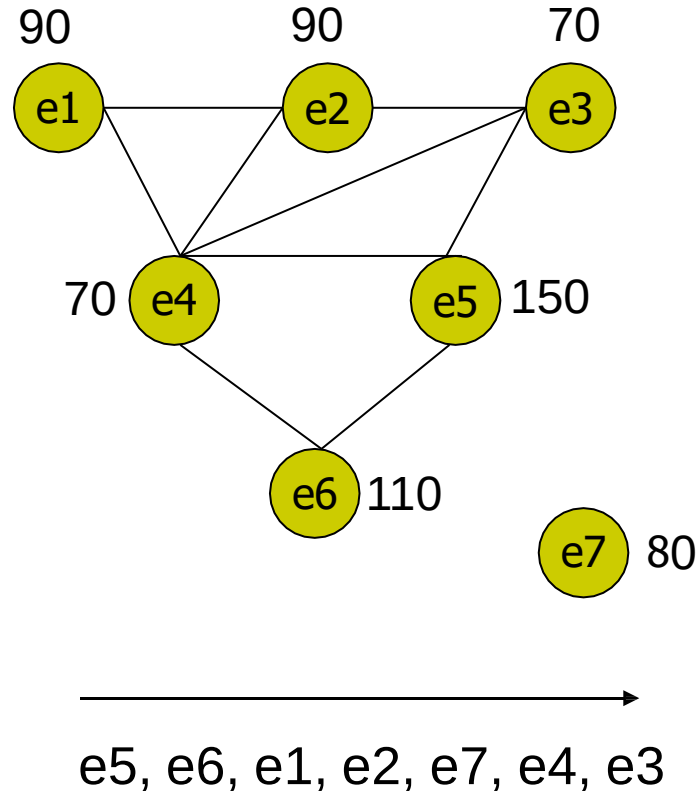
Example: **Largest Degree** Graph Colouring/ Construction/ Ordering Heuristic



Construct a timetable using one event at a time in a given order.



Example: **Largest Enrolment** Graph Colouring/ Construction/ Ordering Heuristic



Construct a timetable using one event at a time in a given order.



Graph-based hyper-heuristics

events

e1	e2	e3	e4	e5	e6	e7	e8	e9	e10	e11	e12	...
----	----	----	----	----	----	----	----	----	-----	-----	-----	-----



heuristic list

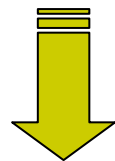
SD	SD	LD	CD	LE	SD	SD	LW	SD	LD	CD	RO	...
----	----	----	----	----	----	----	----	----	----	----	----	-----



Schedule N events using a graph colouring heuristic

order of events

e1	e9	e3	e26	e25	e6	e17	e28	e19	e10	e31	e12	...
----	----	----	-----	-----	----	-----	-----	-----	-----	-----	-----	-----



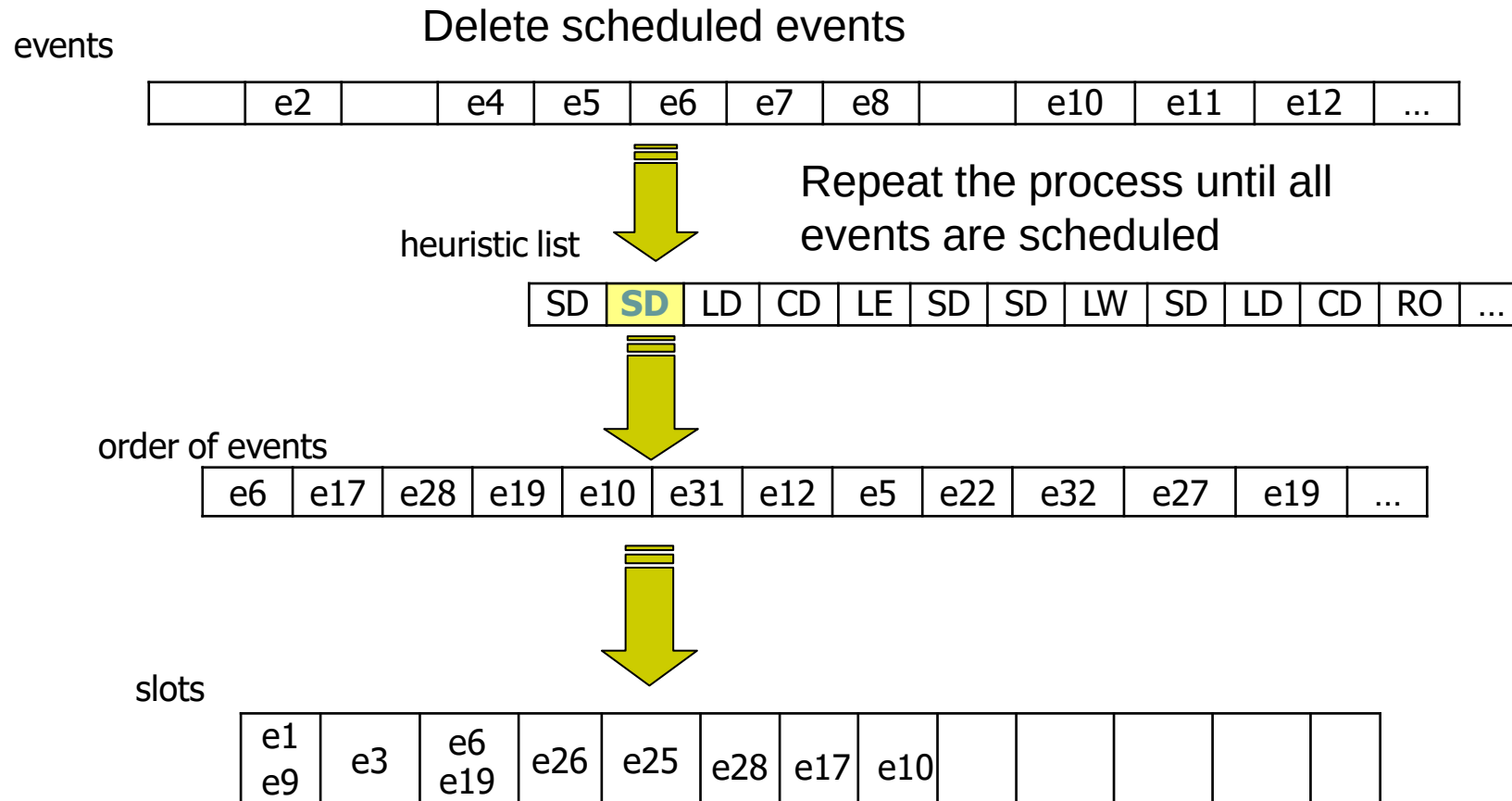
$N=5$

slots

e1	e9	e3	e26	e25								
----	----	----	-----	-----	--	--	--	--	--	--	--	--



Graph-based hyper-heuristics



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Graph-based hyper-heuristics

events

	e2		e4	e5		e7	e8			e11	e12	...
--	----	--	----	----	--	----	----	--	--	-----	-----	-----

heuristic list

SD	SD	LD	CD	LE	SD	SD	LW	SD	LD	CD	RO	...
----	----	----	----	----	----	----	----	----	----	----	----	-----

order of events

e5	e32	e18	e22	e13	e31	e12	e7	e2	e15	e27	e12	...
----	-----	-----	-----	-----	-----	-----	----	----	-----	-----	-----	-----

slots

e1	e3	e6	e26	e25	e28	e17	e10	e5	e32	e13		
e9		e19						e22	e18			

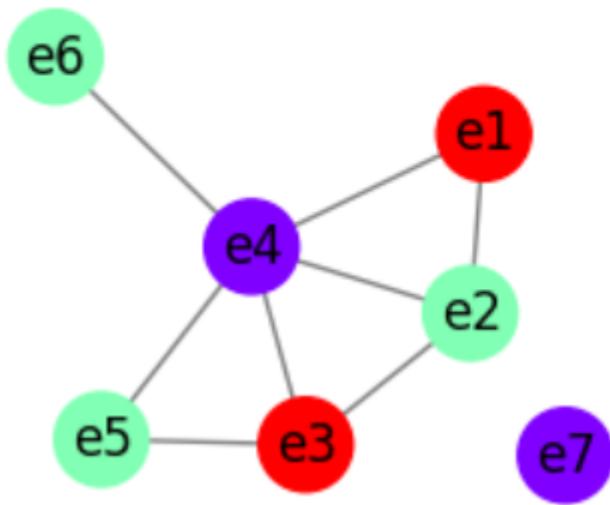
ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Graph-based hyper-heuristics exam timetabling example

Final Color Assignments: {'e4': 1, 'e2': 2, 'e3': 3, 'e1': 3, 'e5': 2, 'e6': 2, 'e7': 1}

Final Graph Coloring



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Graph-based hyper-heuristics

- Tabu Search at the high level (hyper-heuristics)
 - **Neighbourhood operator**: *randomly change two heuristics* in the heuristic list.
 - **Objective function**: quality of solutions built by the corresponding heuristic list.
 - **Tabu list**: visits to the same heuristic lists forbidden.
- Other high-level search strategies tested
 - Steepest Descent
 - Variable neighbourhood search → best performing
 - Iterated Steepest Descent

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Results

Carter's benchmark

Problem	SL	SLR	SCL	SCLR	SCLx	SCLxR	SCLxR(10*)
car91	5.78	5.67	5.52	5.36	5.65	5.43	5.39
car92	4.76	4.68	4.21	4.14	4.53	4.78	4.63
ear83	38.8	38.57	38.68	38.5	37.92	38.22	38.03
hec92	12.35	12.27	12.30	12.81	12.39	12.25	12.11
kfu93	16.21	15.79	15.37	15.23	15.67	15.2	15.12
lse91	12.17	11.36	12.09	11.93	11.56	11.33	11.33
sta83	164.01	163.5	164.06	163.31	158.19	160.19	159.32
tre92	9.15	9.13	8.87	9.08	8.75	9.03	8.97
ute92	29.49	29.17	28.27	28.19	28.01	28.21	28.11
uta93	4.12	4.03	4.05	3.98	3.88	3.95	3.78
york83	44.54	42.67	42.44	42.37	41.37	42.01	41.52

Costs of solutions obtained by GHH upon a different number of heuristics (S: saturation degree, L: largest degree, C: color degree; R: random ordering, Lx: largest weighted)

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



References

- <https://www.geeksforgeeks.org/graph-coloring-applications/>
- <https://towardsdatascience.com/the-graph-coloring-problem-exact-and-heuristic-solutions-169dce4d88ab>
- <https://www.youtube.com/watch?v=L2csXWQMsNg>

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Break





University of
Nottingham

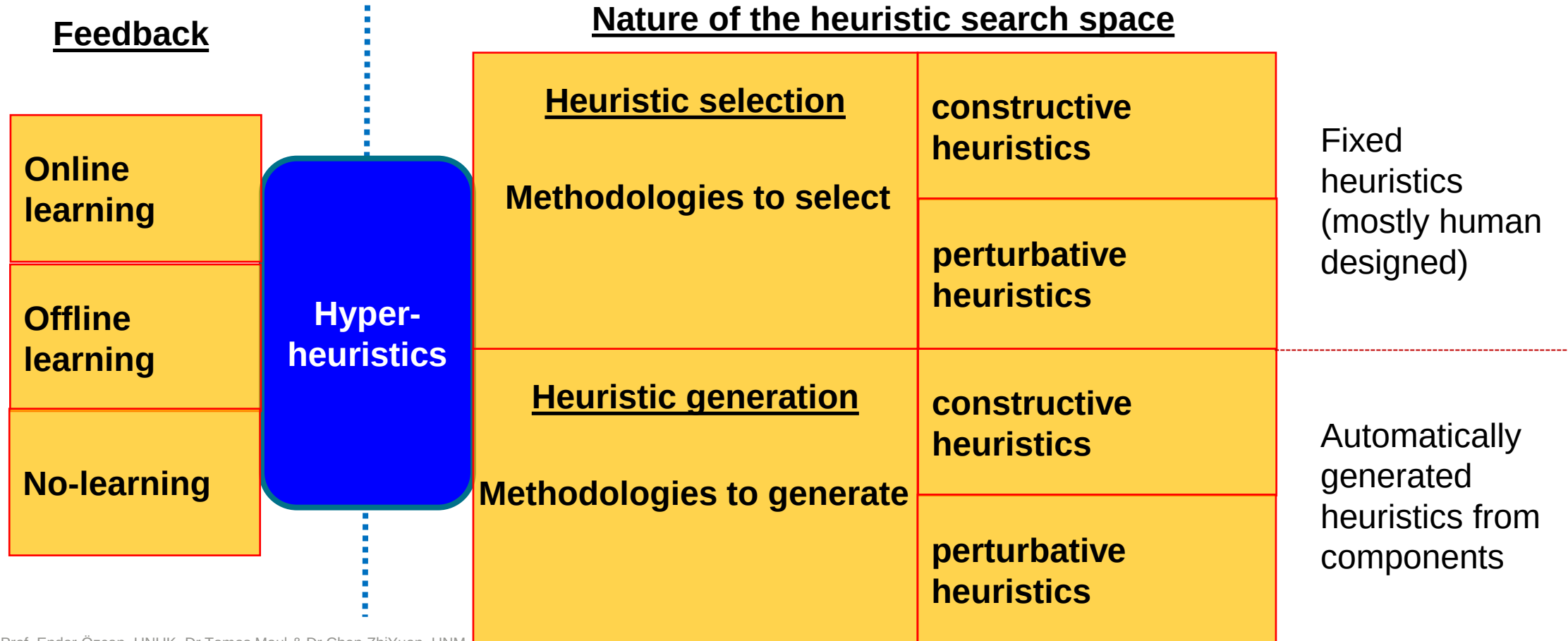
UK | CHINA | MALAYSIA

Generation Hyper-heuristics



Recap: Classification of Hyper-heuristics

E. K. Burke, M. Hyde, G. Kendall, G. Ochoa, E. Özcan, and J. Woodward (2010). A Classification of Hyper-heuristic Approaches. In Gendreau, Michel and Potvin, Jean-Yves (eds.), Handbook of Metaheuristics, International Series in Operations Research & Management Science, Volume 146, pp. 449-468. Springer. [\[PDF\]](#)



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Generation Hyperheuristics

- **Generation Hyper-Heuristics** are a class of hyper-heuristics that create new heuristics or algorithms from basic components rather than selecting existing ones.
- These methods aim to evolve or learn problem-solving strategies rather than just choosing from a predefined set.

Feature	Selection Hyper-Heuristics	Generation Hyper-Heuristics (GP)
Approach	Selects existing heuristics	Generates new heuristics
Learning Process	Learns which heuristics to apply	Evolves new heuristics from components
Example Methods	Choice Function, Reinforcement Learning	Genetic Programming, Grammatical Evolution

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Genetic Programming (GP)

- **Challenge:**

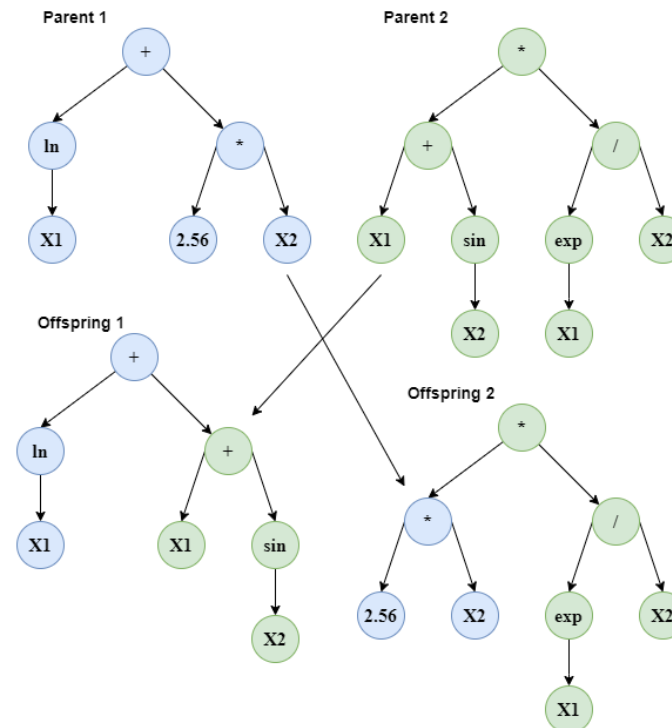
“Get a computer to do what needs to be done, without telling it how to do it.”

- GP provides a method for *automatically creating a working computer program from a high-level problem statement of the problem* (i.e., program synthesis or program induction).
- GP iteratively transforms a population of computer programs into a new generation of programs via evolutionary process.



Why Genetic Programming?

- “It is difficult, unnatural, and overly restrictive to attempt to represent hierarchies of dynamically varying size and shape with fixed length character strings.”
- “For many problems in machine learning and artificial intelligence, the most natural representation for a solution is a computer program.” [Koza, 1994]
- A *parse tree* is a good representation of a computer program for Genetic Programming.



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Tree-Based GP

- In traditional GP, programs are represented as syntax trees, where *internal nodes* are *functions* (operators) and *leaf nodes* are terminals (*constants* or *variables*).
- These trees are evaluated *recursively*, following a *depth-first traversal*.



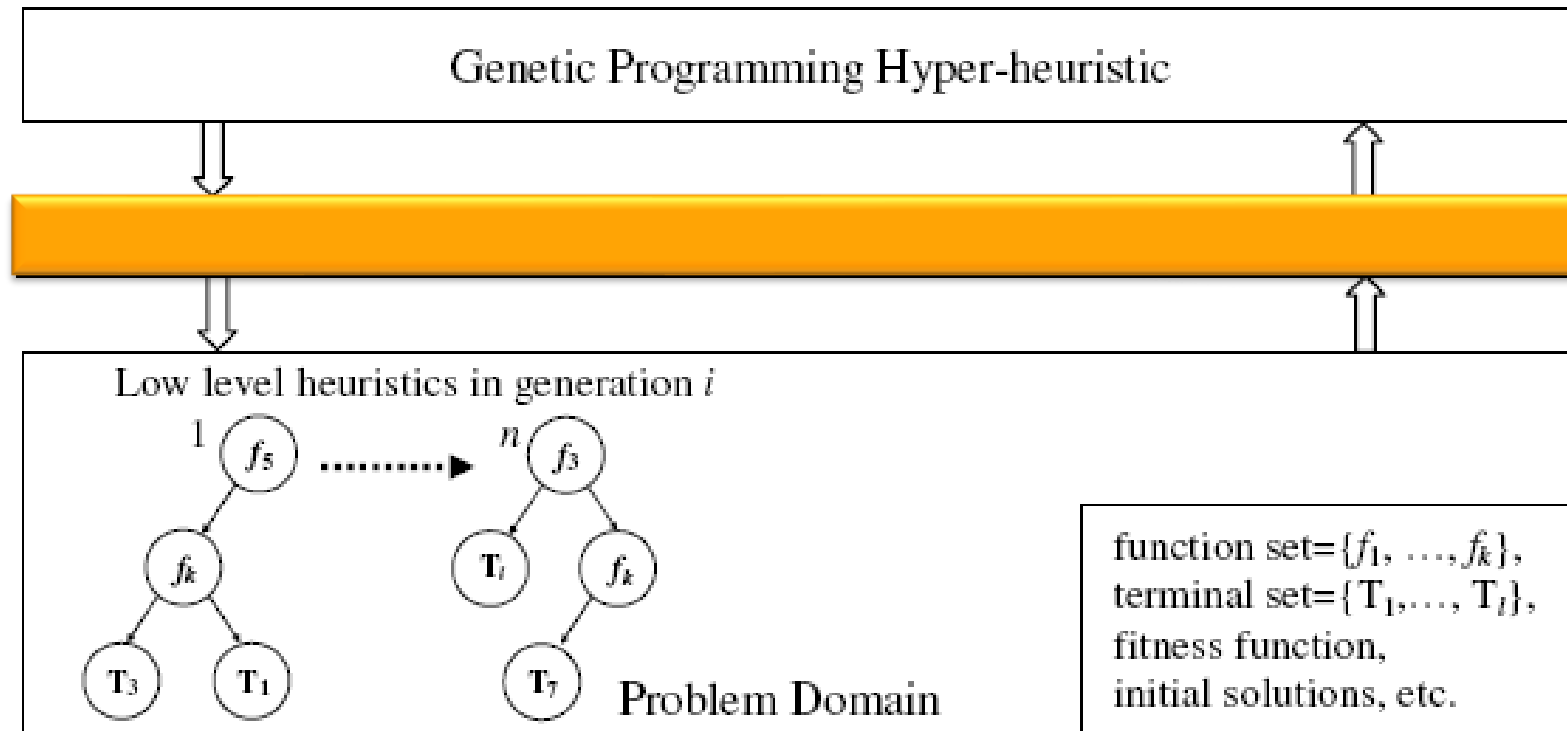
Genetic Programming (GP)

- **Genetic Programming (GP)** is generally considered a type of **Generation Hyper-Heuristic**.
- GP evolves *programs* (which can be *heuristics*) using principles of evolutionary algorithms (genetic), such as *selection*, *crossover*, and *mutation*.
- Instead of simply selecting heuristics, GP **constructs new heuristics** dynamically by evolving mathematical expressions, decision trees, or algorithmic rules.
- GP can be applied to automatically generate heuristics for combinatorial optimization problems, scheduling, MAX-SAT solving, etc.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



A Generation Hyper-heuristic Framework



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Steps in GP

1. Initialization:

- Randomly generate an initial population of program trees.

2. Evaluation:

- Compute the fitness of each program in the population.

3. Selection:

- Select the fittest individuals for reproduction based on the fitness function.

4. Genetic Operations:

- Apply crossover, mutation, and reproduction to create a new population.

5. Replacement:

- Replace the old population with the new one.

6. Iteration:

- Repeat steps 2–5 for several generations.

7. Output:

- The best program (solution) found during the process.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Mai



Key Components of GP

- **Representation:**

- Programs or solutions are typically represented as trees, where:
- Internal nodes represent operators (e.g., +, -, *, /, if).
- Leaves represent operands (e.g., constants, variables).

- **Population:**

- GP maintains a population of candidate solutions, each represented as a program tree.

- **Fitness Function:**

- Evaluates how well a program performs the given task.
- Example: For a symbolic regression task, fitness might measure the error between the program's output and the target values.

ACK: Prof. Ender Özcan, UNL, Dr. Thomas Mautz & Dr. Christian Bittmann, UNL



Key Components of GP

- **Genetic Operators:**
 - **Selection:** Chooses parent solutions based on fitness (e.g., tournament selection, roulette wheel).
 - **Crossover:** Combines subtrees from two parent solutions to create offspring.
 - **Mutation:** Randomly alters parts of a program tree to introduce diversity.
 - **Reproduction:** Copies solutions directly to the next generation.
- **Termination:**
 - The process ends when a predefined number of generations is reached or when a satisfactory solution is found.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Genetic Operations

- GP is an *evolutionary algorithm* containing the same algorithmic components, including:
 - **Random generation** of the initial population of possible solutions (programs).
 - Genetic **crossover** of two promising solutions to create new possible solutions (programs).
 - **Mutation** of promising solutions to create new possible solutions (programs).

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



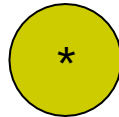
Randomly Generating Programs

- Randomly generate a program that takes two (or more) arguments and uses basic arithmetic to return an answer.
 - Function set = $\{+, -, *, /\}$
 - Terminal set = $\{\text{integers}, X, Y\}$
- Randomly select either a function or a terminal to represent our program.
- If a function was selected, recursively generate random programs to act as arguments.



Randomly Generating Programs

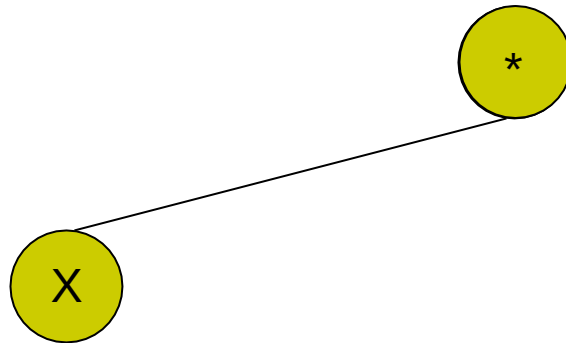
(* ...) – assume multiplication with four arguments: $\text{arg1} + \text{arg2} + \text{arg3} + \text{arg4}$
randomly create each argument one by one





Randomly Generating Programs

arg1 is a terminal, so stop
(* X ...)

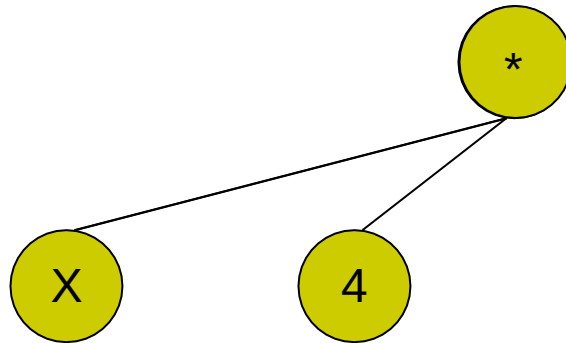


ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Randomly Generating Programs

arg2 is terminal stop (*
X 4 ...)



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

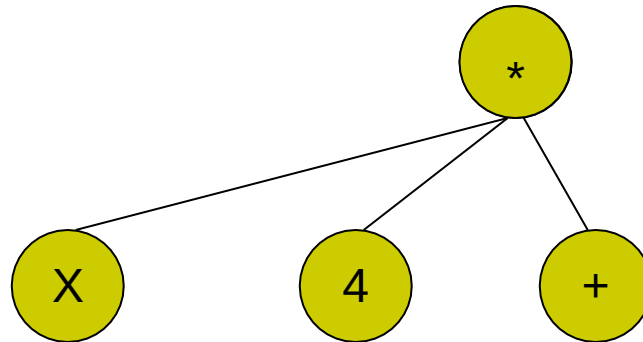


Randomly Generating Programs

arg3 is a function

($*$ X 4 (+ ...) ...)

assume addition requires two arguments, since a function is selected, recursively generate random programs to act as arguments.

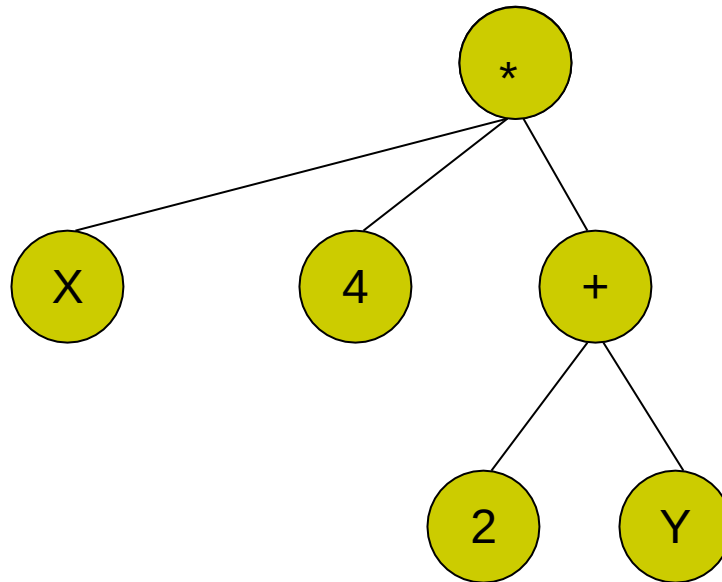




Randomly Generating Programs

(* X 4 (+ 2 Y) ...)

arguments of addition are terminals, so stop recursion for each



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

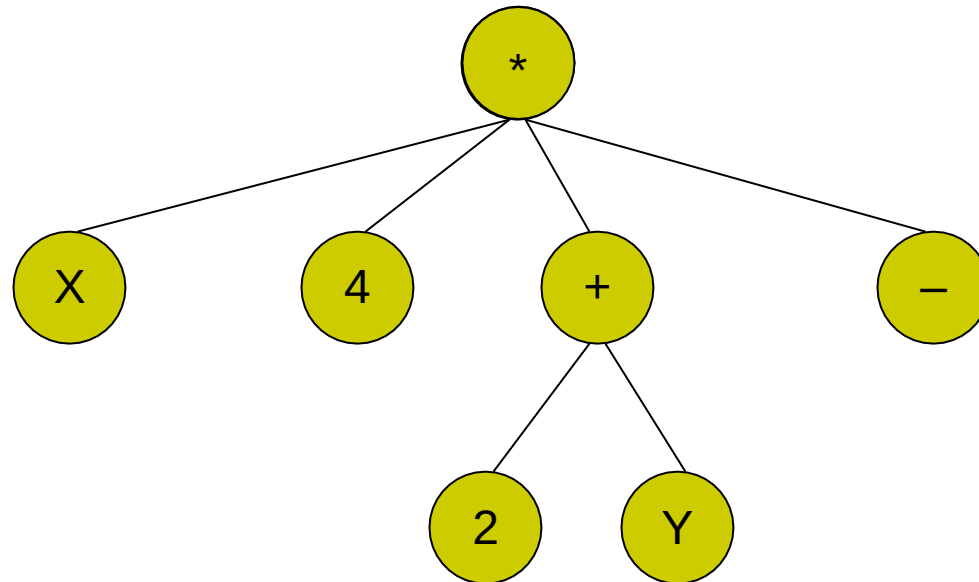


Randomly Generating Programs

arg3 is a function

$(* X 4 (+ 2 Y) (- \dots))$

assume subtraction requires two arguments, since a function is selected, recursively generate random programs to act as arguments



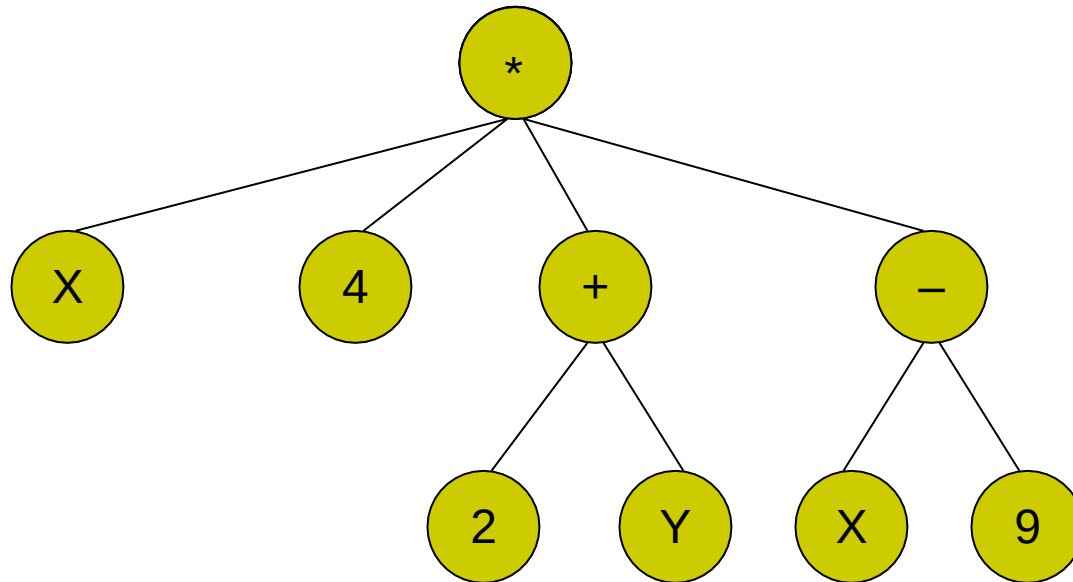
ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Randomly Generating Programs

$(* X 4 (+ 2 Y) (- X 9))$

arguments of division are terminals, so stop recursion for each



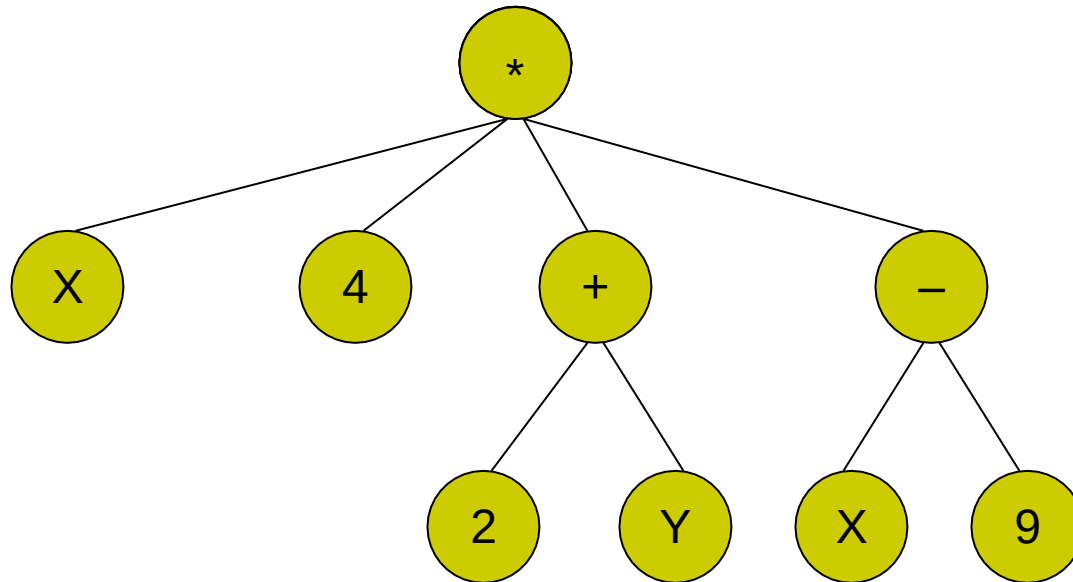
ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Mutation

First pick a random node

$(* X 4 (+ 2 Y) (- X 9))$

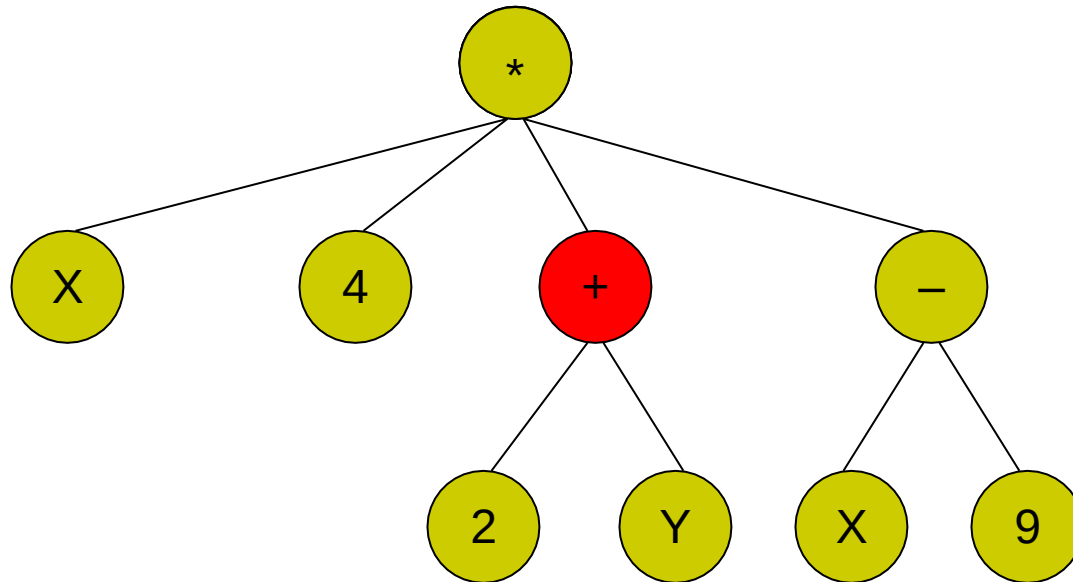




Mutation

First pick a random node

(* X 4 (+ 2 Y) (- X 9))



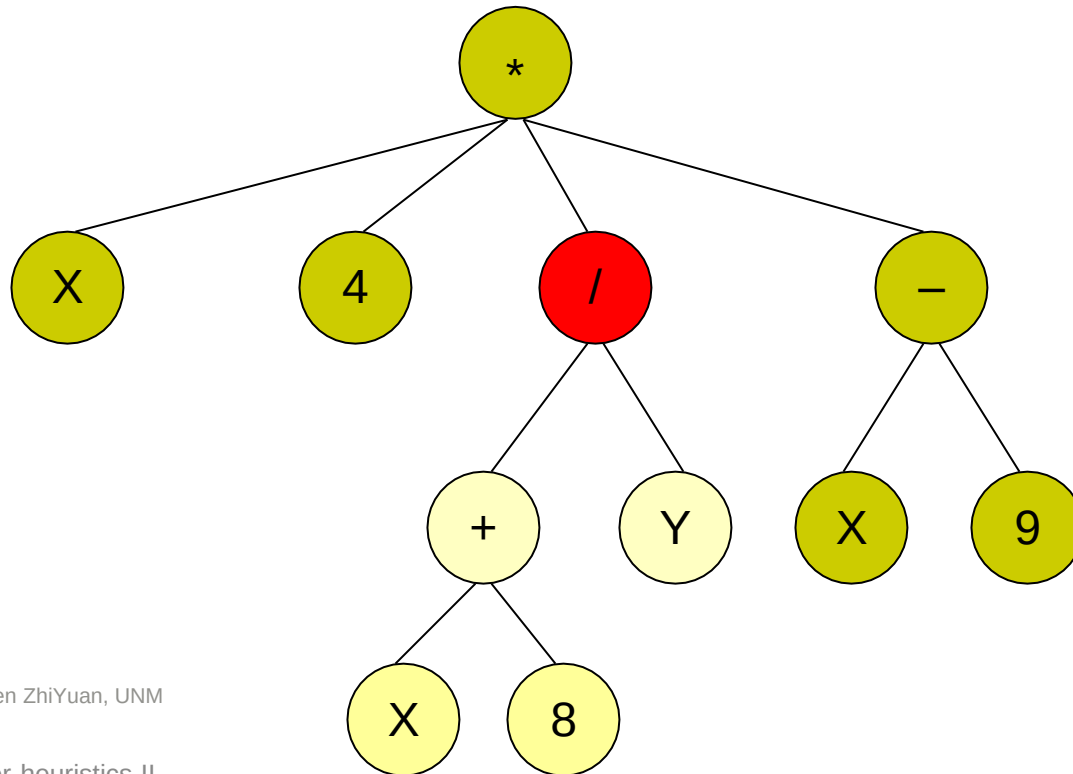
ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Mutation

(* X 4 (\ (+ X 8) Y) (- X 9))

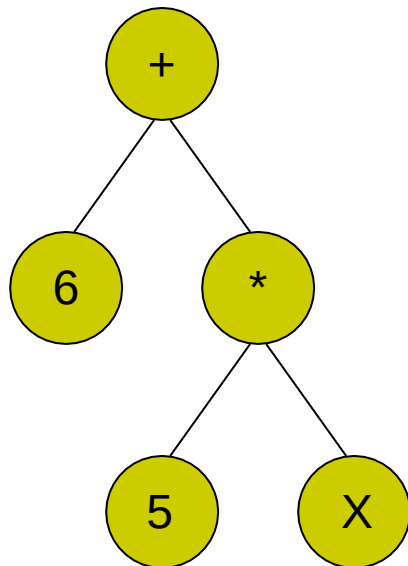
Delete the node and its children and replace with a randomly generated program.





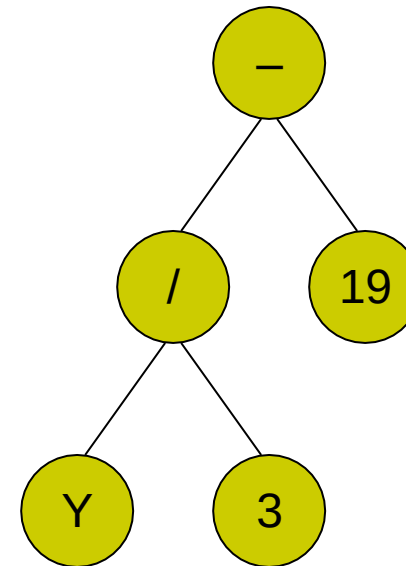
Crossover

$(+ 6 (* 5 X))$



Pick a random node in each program

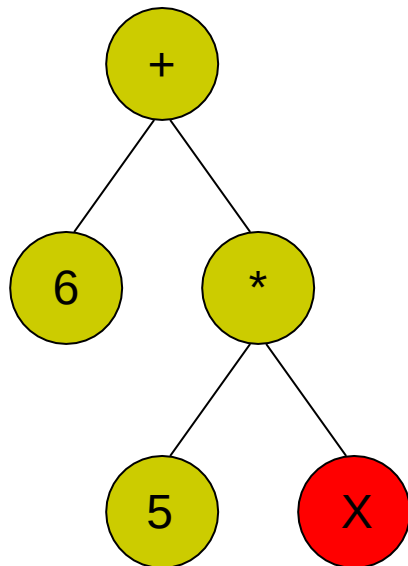
$(- (/ Y 3) 19)$





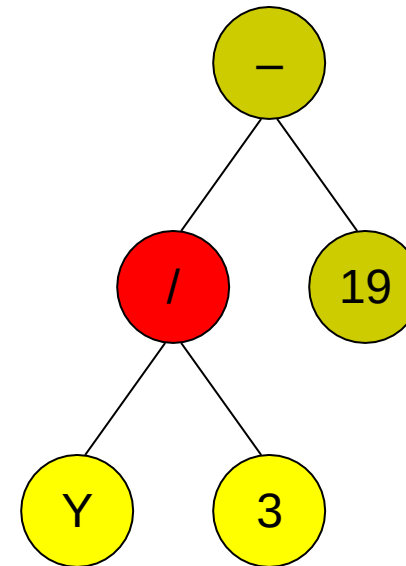
Crossover

(+ 6 (* 5 X))



Swap the two nodes

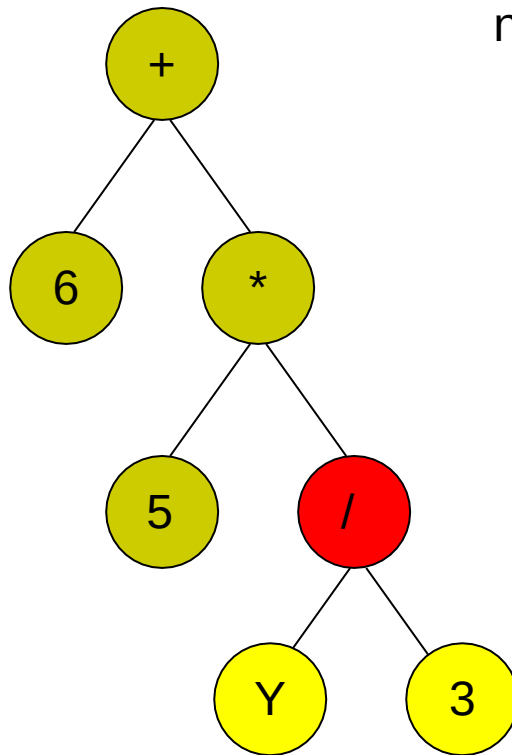
(- (/ Y 3) 19)





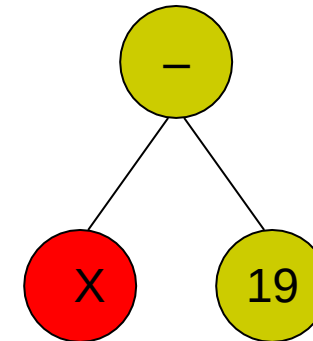
Crossover

(+ 6 (* 5 (/ Y 3)))



Now we have two offspring/
new candidate solutions

(- X 19)



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



A Computer Program in C

```
int foo (int time)
{
    int temp1, temp2;
    if (time > 10)
        temp1 = 3;
    else
        temp1 = 4;
    temp2 = temp1 + 1 + 2;
    return (temp2);
}
```

What would be the output of this code as time changes from 0 to 14?



Generating Outputs for time = 1 to time = 14:

time	temp1	temp2 (return value)
1	4	7
2	4	7
3	4	7
4	4	7
5	4	7
6	4	7
7	4	7
8	4	7
9	4	7
10	4	7
11	3	6
12	3	6
13	3	6
14	3	6

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



A Computer Program in C

- If we have the following observations, can we *reverse engineer* the mapping function?

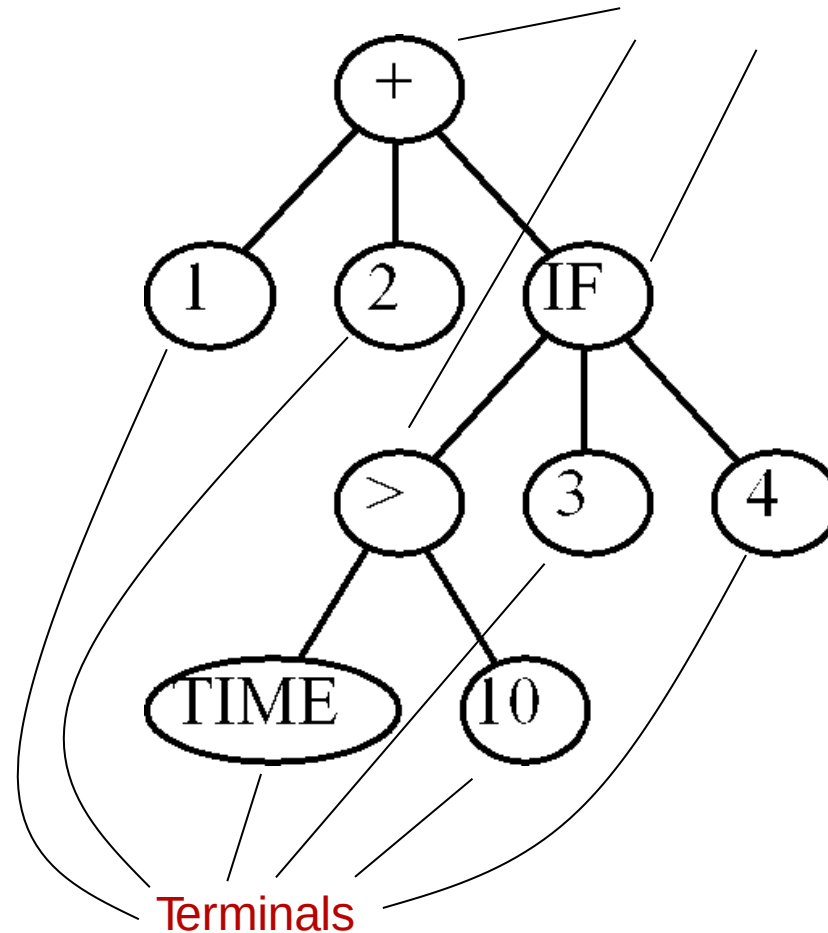
Time	Output
0	7
1	7
2	7
3	7
4	7
5	7
6	7
7	7
8	7
9	7
10	7
11	6
12	6
13	6
14	6

A plausible step-function representation is:

$$f(x) = \begin{cases} 7, & x \leq 10 \\ 6, & x > 10 \end{cases}$$



Using Trees To Represent Computer Programs Functions



(+ 1 2 (IF (> TIME 10) 3 4))

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Using Trees To Represent Computer Programs Functions

- The GP tree follows the logic:

CSS

$(+ 1 2 (\text{IF } (> \text{TIME } 10) 3 4))$

- If $\text{TIME} > 10$, IF returns 3, so $1 + 2 + 3 = 6$
- Else, IF returns 4, so $1 + 2 + 4 = 7$

```
def gp_program(time):  
    """Evaluates the expression tree for a given time input.  
    if time > 10:  
        temp1 = 3  
    else:  
        temp1 = 4  
    return 1 + 2 + temp1  # Equivalent to (+ 1 2 (IF (> TIME  
  
# Run for inputs from 0 to 14  
results = {time: gp_program(time) for time in range(15)}  
  
# Print results  
for time, output in results.items():  
    print(f"time = {time}, output = {output}")
```



Some Software Libraries for GP

- Python

- DEAP: <https://deap.readthedocs.io/en/master/>

- Java

- ECJ: <http://cs.gmu.edu/~eclab/projects/ecj/>
 - GEVA (grammatical evolution): <http://ncra.ucd.ie/Site/GEVA.html>



Steps for Reverse Engineering the Function using GP

1. Define the **primitive set** (basic operations like $+$, $-$, $*$, $/$).
2. Create an **individual representation** (tree-based structure).
3. Define a fitness function that evaluates how well an evolved function approximates the given dataset.
4. Use evolutionary operators (mutation, crossover, selection) to evolve the best-fitting function.
5. Run the GP and extract the best symbolic function.

■ Implementation Using DEAP

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Evolved Mapping Function 1 based on DEAP vs Ground Truth

Ground Truth:

(+ 1 2 (IF (> TIME 10) 3 4))

The **GP-evolved function** approximates this with:

$$f(x) = \begin{cases} 7, & x \leq 10 \\ 10 - \frac{3x}{10}, & x > 10 \end{cases}$$

GP tends to overcomplicate simple thresholds: The actual function is likely piecewise.



More Evolved Mapping Functions based on DEAP

1. `min(7, min(protected_div(max(mul(7, max(10, 10))), protected_div(9, 10)), max(min(add(mul(add(min(max(-5, 8), 10), -2), min(10, -1)), add(add(10, -5), add(10, 3))), ARG0), min(10, 10))), 10))`
2. `add(protected_div(sub(min(min(ARG0, ARG0), add(5, max(sub(10, mul(mul(max(protected_div(6, protected_div(6, add(5, 2))), mul(ARG0, 10))), protected_div(add(5, sub(mul(protected_div(10, ARG0), min(10, 10))), mul(protected_div(6, add(5, 2)), min(ARG0, ARG0)))), sub(10, ARG0))), -8)), sub(protected_div(6, 10), 8))), protected_div(ARG0, max(max(sub(mul(protected_div(add(10, mul(protected_div(6, add(5, 2))), min(ARG0, ARG0))), protected_div(ARG0, 8)), sub(min(10, ARG0), sub(10, 10))), protected_div(10, ARG0)), sub(10, protected_div(max(max(sub(mul(protected_div(add(10, mul(protected_div(6, add(5, 7))), min(ARG0, ARG0))), protected_div(ARG0, 8)), sub(min(10, ARG0), sub(10, 10))), protected_div(10, ARG0)), sub(10, 8)), 6), add(10, 10))), 6))), ARG0, 6)`

ACK: Prof. Ender Özcan, UNIK, Dr. Thomas Maul & Dr. Chen Zhi-Kun, UNIK



Some selected real-world applications of Genetic Programming

- Automated design of mechatronic systems (NSF)
 - Climatology: Estimation of heat flux between the atmosphere and sea ice, modelling global temperature changes
 - Clinical decision support in ophthalmology
 - Container loading optimisation
 - Scheduling, timetabling
 - Machine learning
 - Vehicle routing ...
-
- GP is commonly used to solve optimization, symbolic regression, classification, and automated design problems.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



University of
Nottingham

UK | CHINA | MALAYSIA

Genetic Programming for Bin Packing



1D Offline Bin Packing

Pack a **set** of items of sizes s_i for $i = 1, \dots, n$

- Sizes are integer values and $s_i \in [1, C]$
- C is the fixed capacity of each bin

in such a way that

- Never exceed bin capacity.
- Minimise number of bins used.

Standard NP-hard problem

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Traditional bin-packing algorithms

Traditional bin-packing algorithms (e.g., First Fit, Best Fit) rely on predefined heuristic rules.

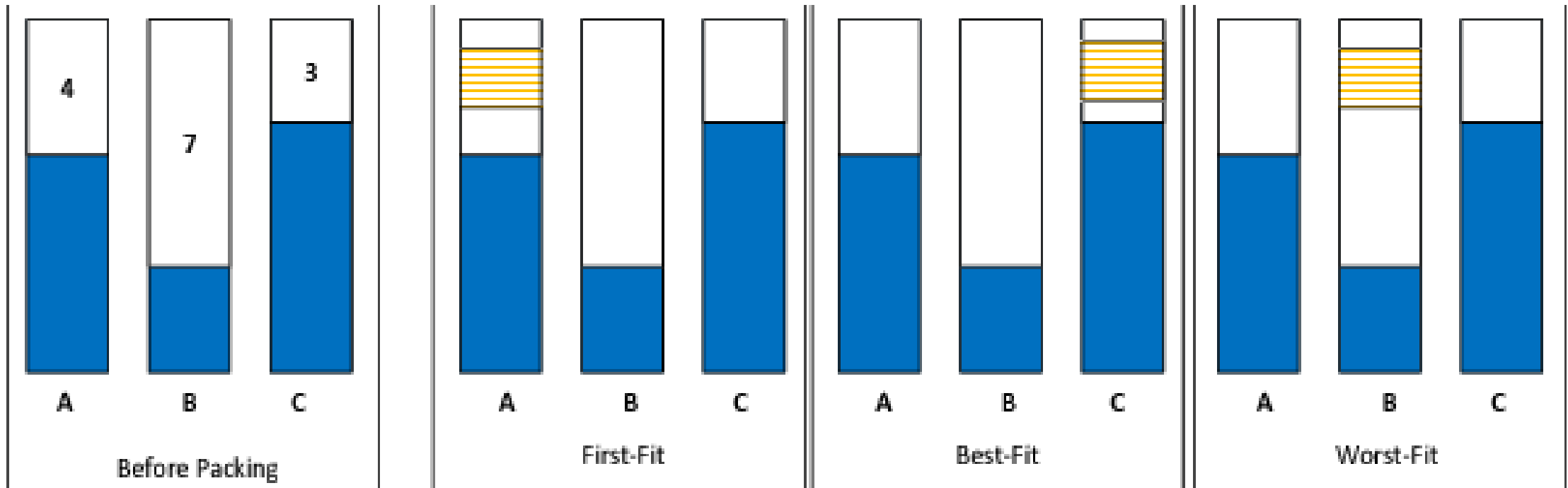


Fig. 2: Example illustrating the bin packing algorithms.



Traditional bin-packing algorithms

Heuristic	Strategy	Time Complexity	Bin Efficiency	Speed	Notes
First Fit (FF)	Place item in the first available bin that has space	$O(n \log n)$ (if sorted)	Moderate	Fast	Simple, but can leave gaps
First Fit Decreasing (FFD)	Sort items in descending order, then apply First Fit	$O(n \log n)$	Good	Moderate	Often used in practice for better results
Best Fit (BF)	Place item in the bin with the tightest remaining space	$O(n \log n)$	Good	Slower	Minimizes wasted space but increases time complexity
Best Fit Decreasing (BFD)	Sort items in descending order, then apply Best Fit	$O(n \log n)$	Very Good	Slower	Generally performs better than BF
Next Fit (NF)	Place item in the current bin if it fits; otherwise, start a new bin	$O(n)$	Poor	Fastest	Avoids scanning previous bins, but often uses more bins
Worst Fit (WF)	Place item in the bin with the most remaining space	$O(n \log n)$	Poor	Moderate	Leaves large gaps, usually inefficient
Worst Fit Decreasing (WFD)	Sort items in descending order, then apply Worst Fit	$O(n \log n)$	Moderate	Slower	Improves upon WF but still not optimal

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul &



Genetic Programming-Based Hyperheuristics

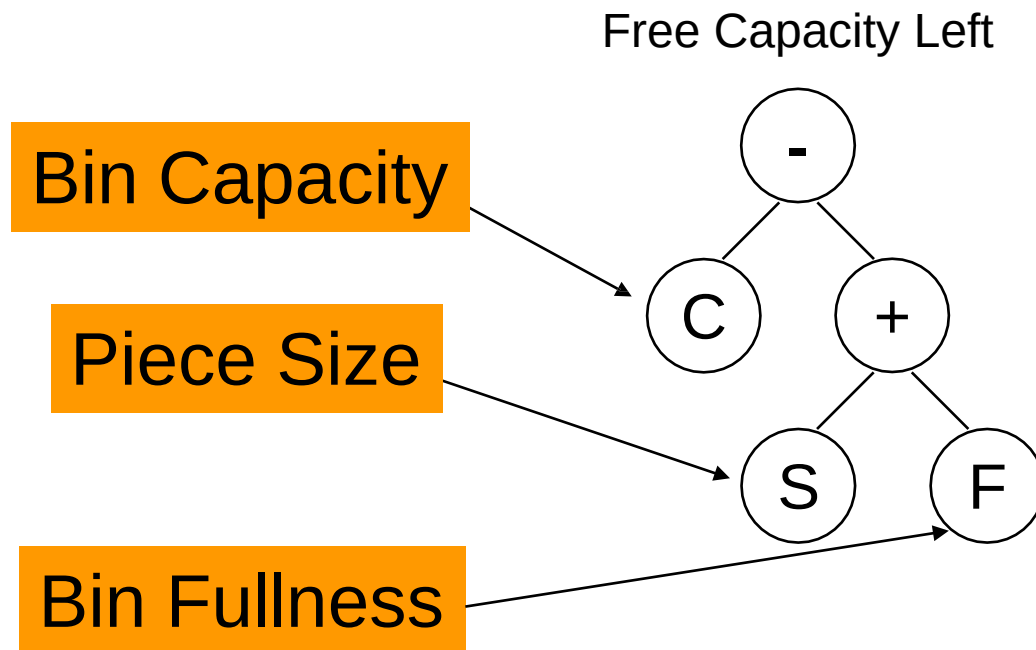
- **Genetic Programming (GP)** can be used to automatically discover (generate) *new heuristic* rules by combining building blocks of existing heuristics or mathematical operations.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Genetic Programming

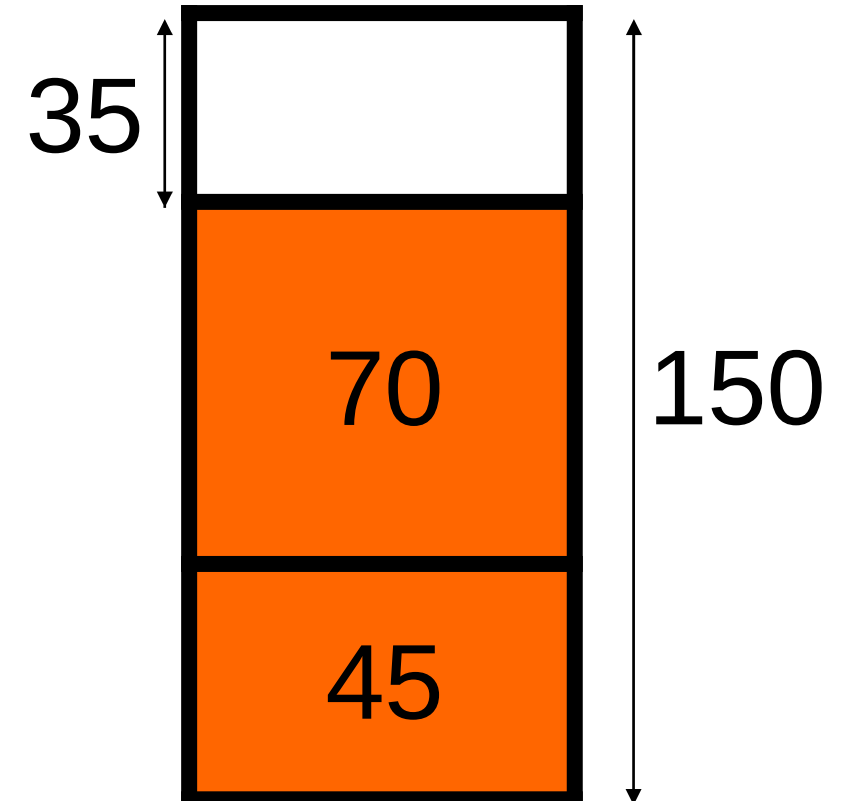
- Evolves trees representing a program.
- Following tree is a program that calculates the space left at the top of the bin.
- Train and test.



Bin Capacity, $C = 150$

Piece Size = S

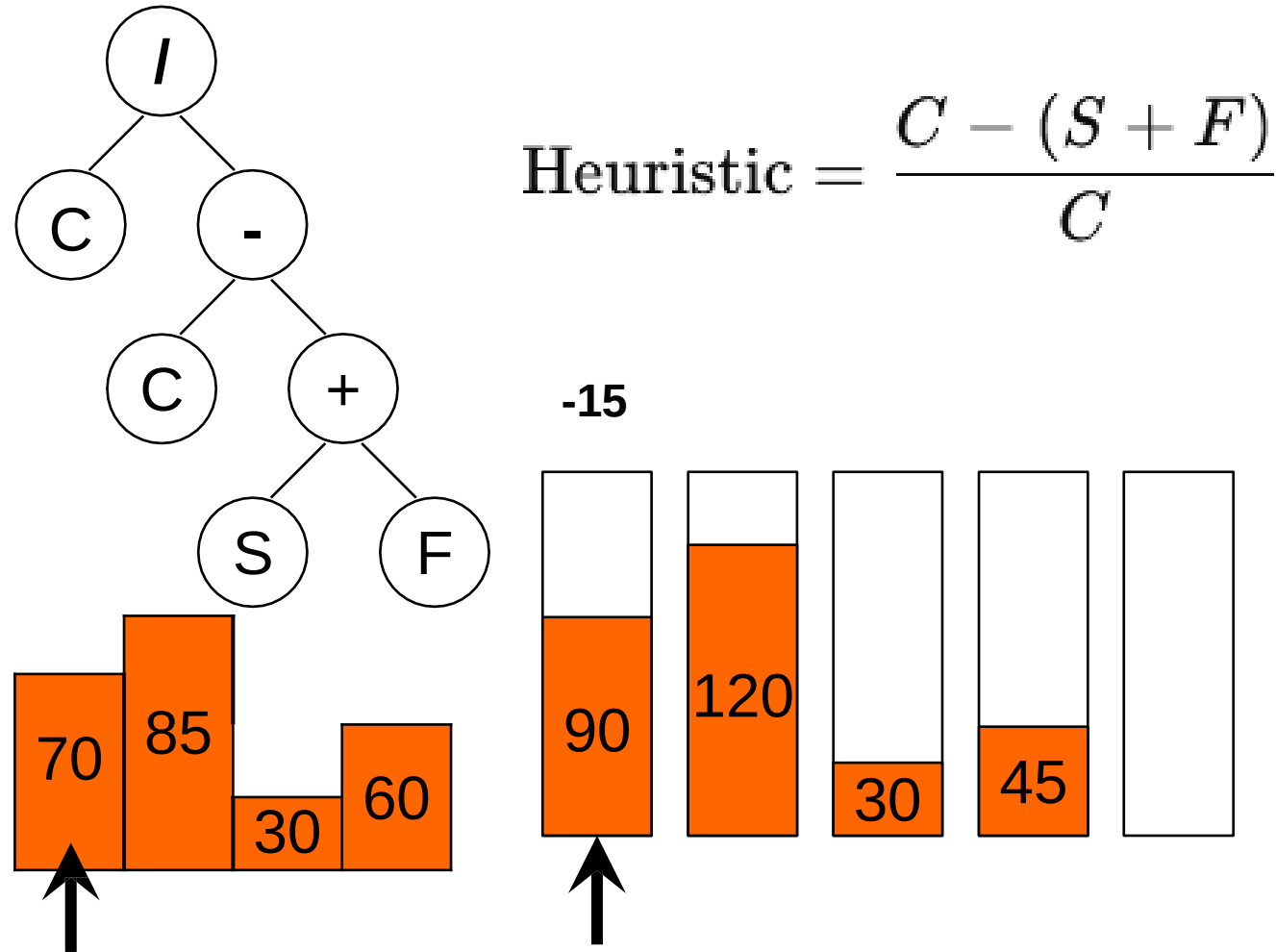
Bin Fullness, $F = 70 + 45 = 115$



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



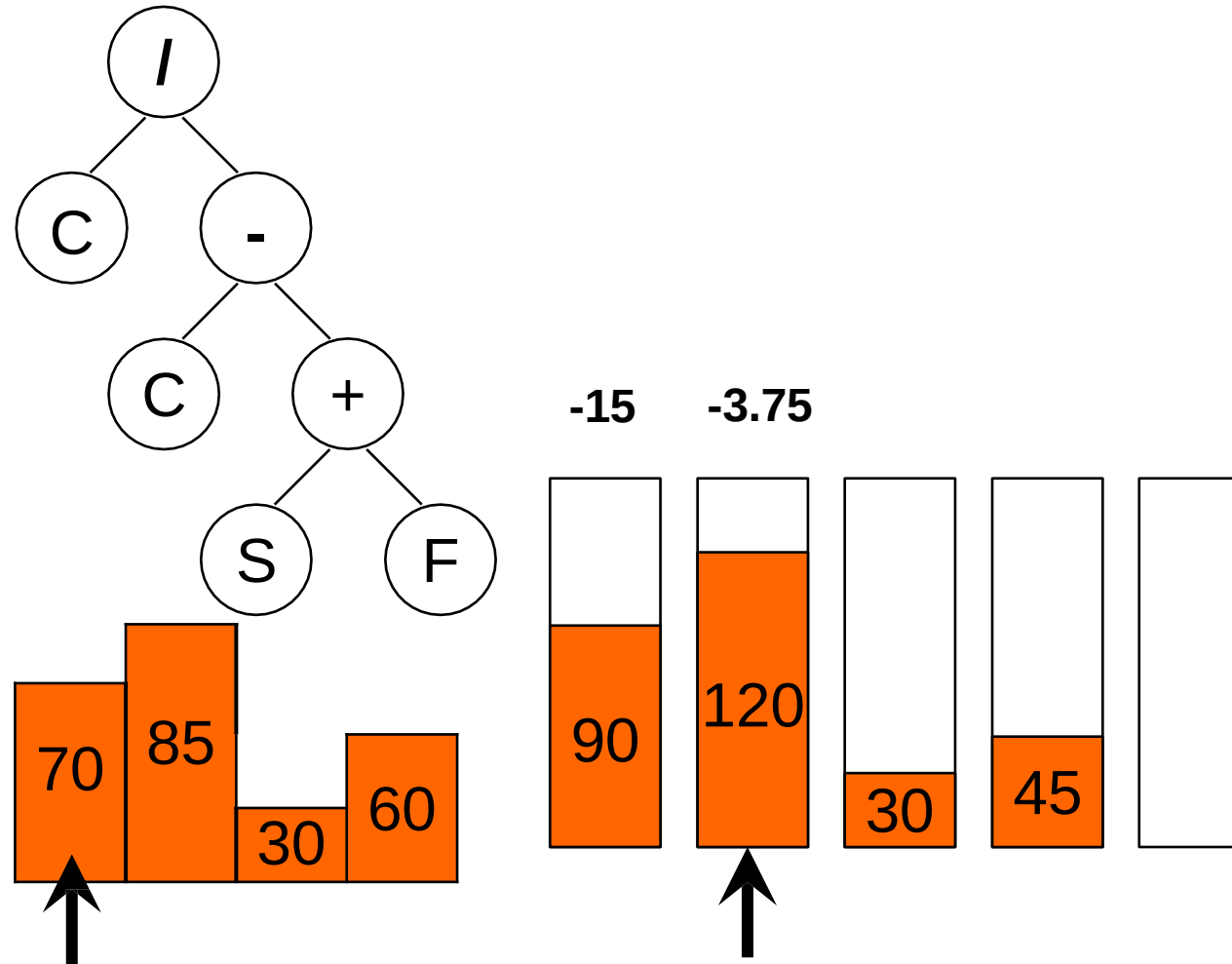
Genetic Programming Heuristics – Bin Packing



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



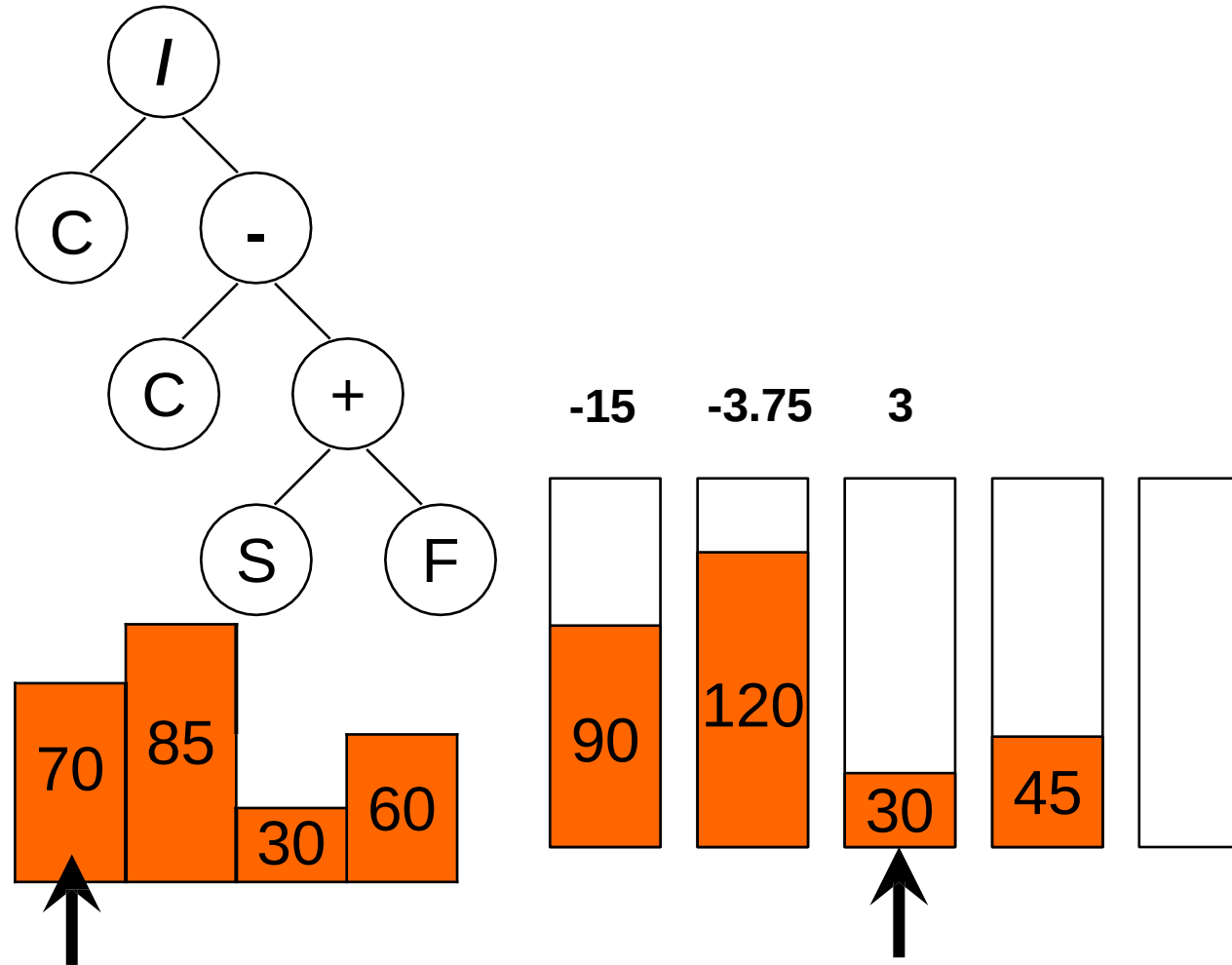
Genetic Programming Heuristics – Bin Packing



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



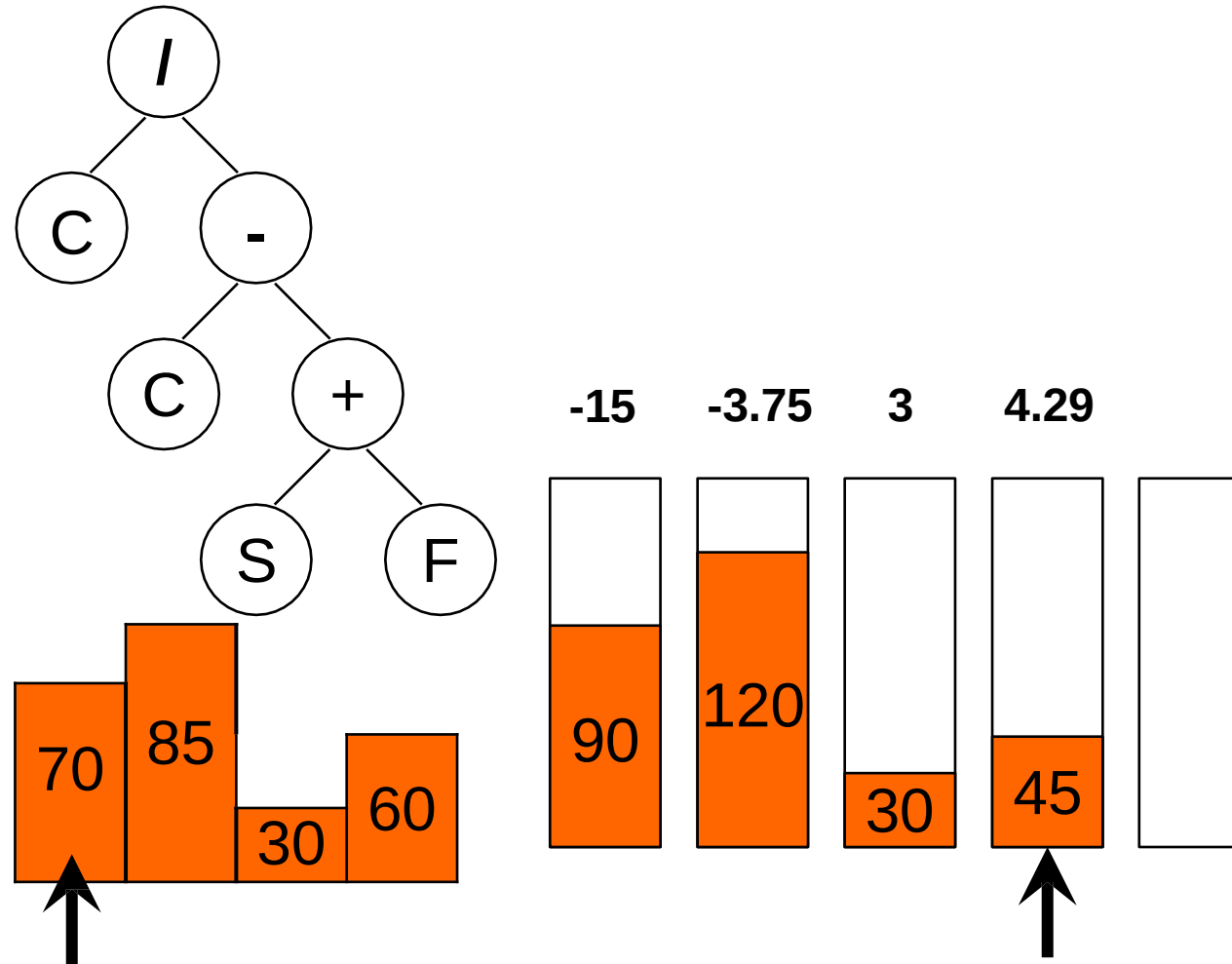
Genetic Programming Heuristics – Bin Packing



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



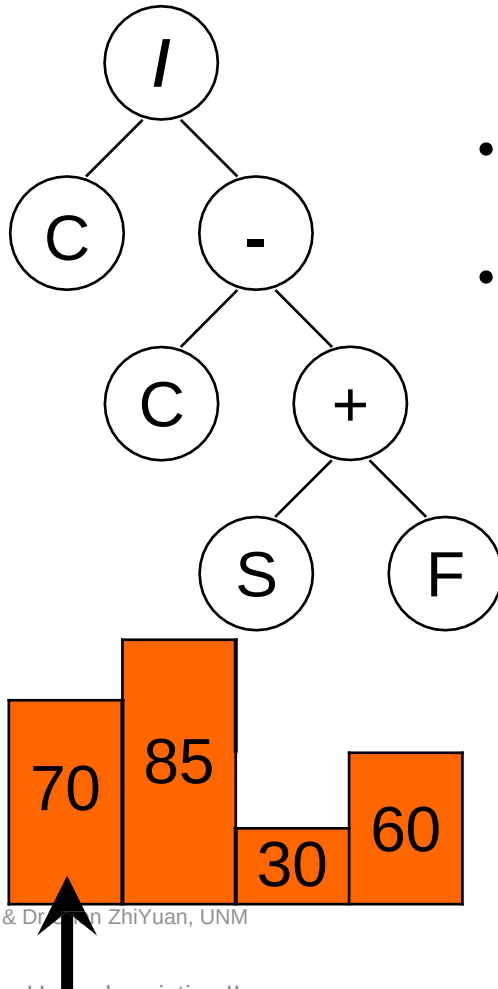
Genetic Programming Heuristics – Bin Packing



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

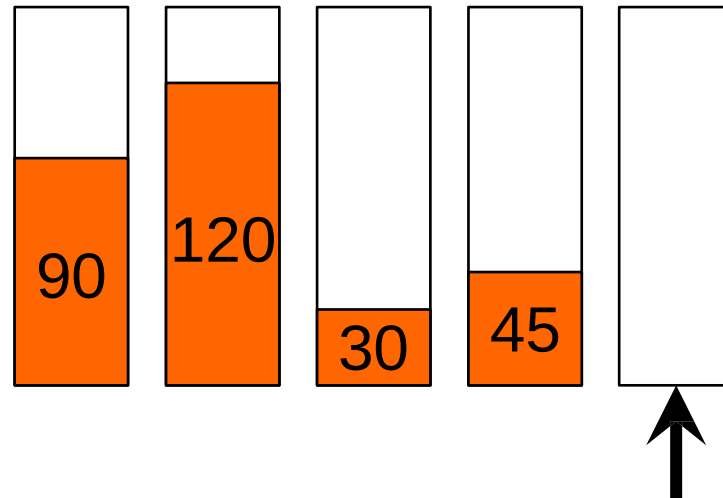


Genetic Programming Heuristics – Bin Packing



- The values like -15, -3.75, 3, 4.29, and 1.88 are outputs of the GP-evolved heuristic function, which is based on bin properties (capacity, space remaining, number of items, etc.).
- Lower scores (negative values) may indicate bins that are nearly full (e.g., bins with 90 or 120 filled).
- Higher scores (e.g., 4.29) suggest that the bin is a better candidate for placing the next item.

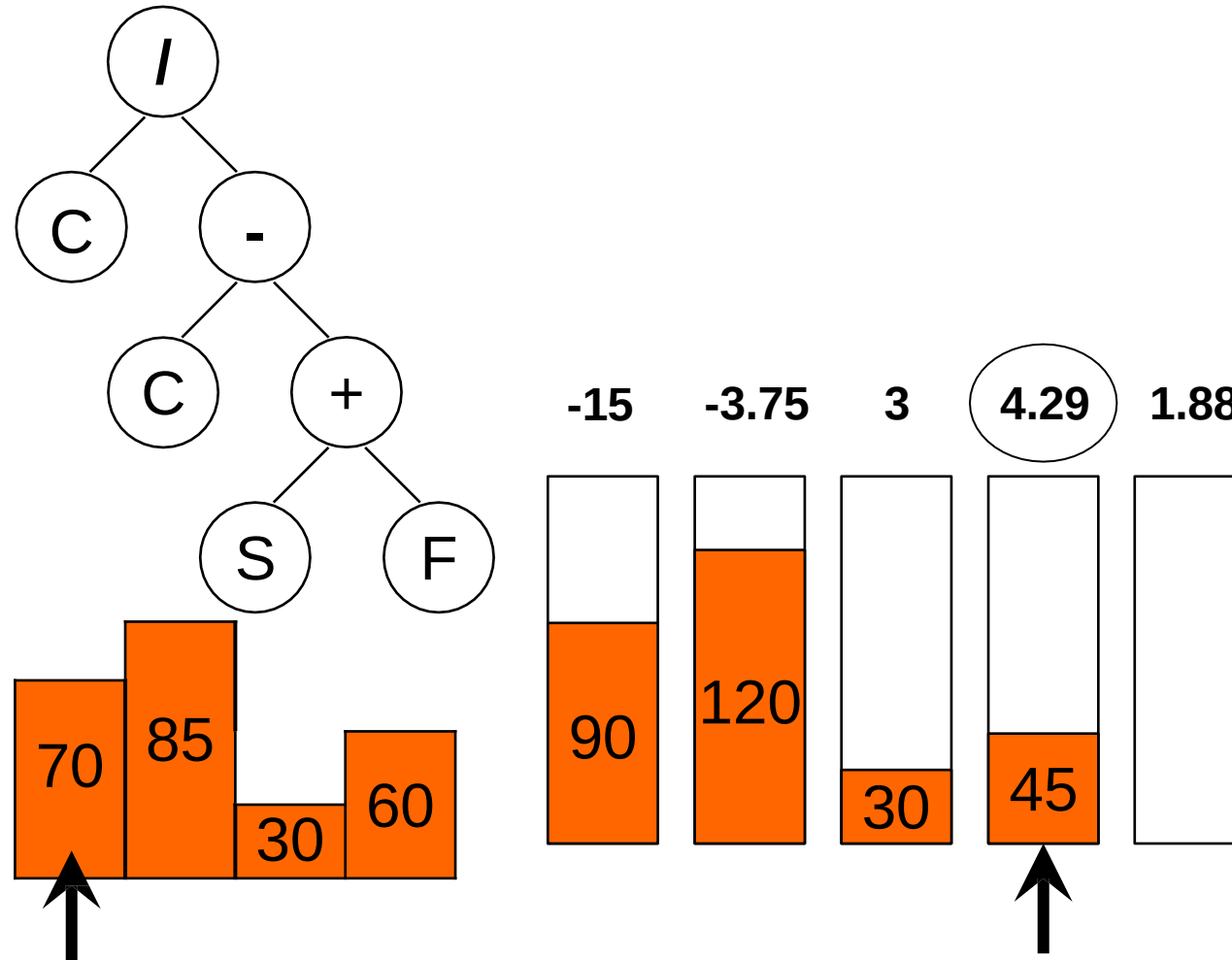
-15 -3.75 3 4.29 1.88



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr. ZhiYuan, UNM



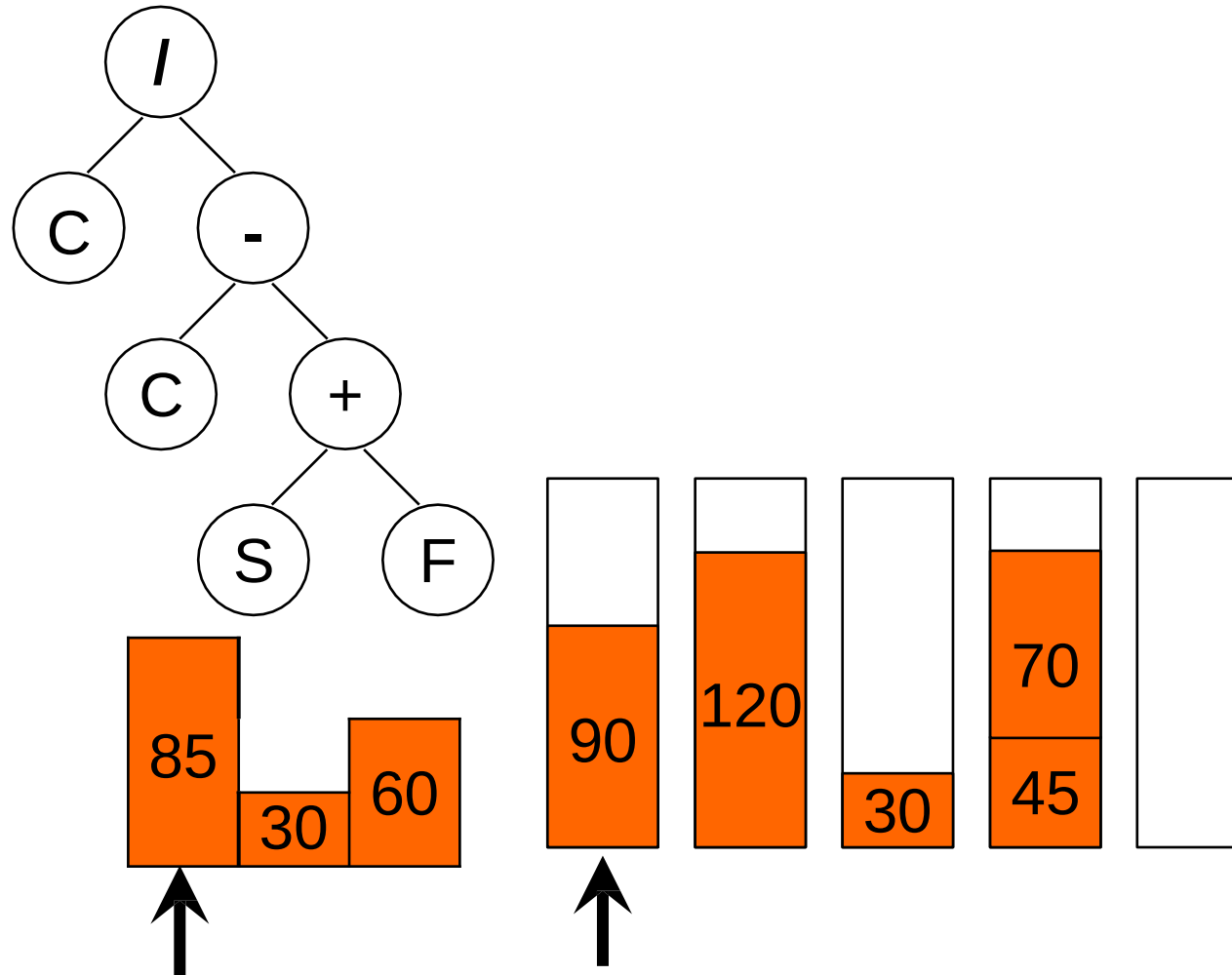
Genetic Programming Heuristics – Bin Packing



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Genetic Programming Heuristics – Bin Packing



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Example of an evolved heuristics function

```
# Define the problem: Bin Packing
bin_capacity = 150 # Each bin has a maximum capacity of 150
items = [70, 85, 30, 60, 90, 120, 30, 45] # List of items to pack into bins
```

The best-evolved function found through Genetic Programming (GP) for the **Bin Packing Problem (BPP)** is:

$$\text{add}(\text{mul}(\text{ARG0}, \text{ARG1}), \text{sub}(\text{ARG2}, -1.7638659770981846))$$

Where:

- **ARG0** : Bin capacity (fixed at 150).
- **ARG1** : Space remaining in the bin.
- **ARG2** : Number of items in the bin.
- **-1.76...** : A constant evolved by GP.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Example of an evolved heuristics function

1. `mul(ARG0, ARG1) → Bin Capacity * Space Remaining`

- This term multiplies the bin capacity (150) by the remaining space in a bin.
- If a bin is almost full, the value is small; if a bin has more space, the value is large.
- Encourages filling bins efficiently.

2. `sub(ARG2, -1.7638...) → Number of Items in Bin + 1.76...`

- This slightly increases the weight of bins with more items.
- Might favor bins with more items to avoid fragmentation (fewer half-empty bins).

3. `add(mul(ARG0, ARG1), sub(ARG2, -1.7638...))`

- Combines the two factors into a single score.
- Encourages selecting bins that:
 - Have more space (but are not empty).
 - Have a reasonable number of items.
 - Avoids bins that are too full or too empty.



Example of an evolved heuristics function

How Does This Work as a Heuristic?

1. **Preference for partially filled bins:** It doesn't blindly pick the fullest bin but balances space and item count.
2. **Avoids excessive fragmentation:** The small constant tweak helps it avoid creating too many bins.
3. **Adapts to different item sizes:** By using multiplication and addition, it dynamically adjusts decisions.

Expected Behavior in Packing

- It will likely **prefer bins that have just enough space** rather than always picking the fullest or emptiest.
- If two bins have space, it **slightly favors** the one with more items.



Steps for Applying GP Hyperheuristics to BPP

1. Define Building Blocks:

- Use basic operations (e.g., `+`, `-`, `/`, `*`).
- Incorporate problem-specific functions (e.g., "remaining bin space," "item size").
- Example features for the BPP:
 - `RemainingBinSpace` : Space left in the current bin.
 - `ItemSize` : Size of the item being packed.
 - `BinsUsed` : Number of bins currently in use.

2. Initialize Population:

- Generate a population of random program trees. Each tree represents a heuristic.
- Example: A tree could represent the rule `RemainingBinSpace - ItemSize`.

3. Evaluate Fitness:

- Apply each heuristic to solve multiple BPP instances.
- Fitness is typically based on the number of bins used. Lower bins = better fitness.



Steps for Applying GP Hyperheuristics to BPP

4. Genetic Operations:

- **Crossover:** Combine subtrees of two parent heuristics.
- **Mutation:** Randomly alter parts of a program tree.
- **Reproduction:** Copy the best heuristics unchanged to the next generation.

5. Termination:

- Stop after a fixed number of generations or when the fitness improvement stalls.
- Return the best heuristic found.



Example: GP Hyperheuristics for BPP

Problem Setup

Consider a set of items with sizes: $\{0.4, 0.7, 0.2, 0.5, 0.6, 0.3\}$.

Building Blocks

- **Features:**
 - **ItemSize:** The size of the current item.
 - **RemainingBinSpace:** Space left in the current bin.
- **Operations:**
 - **Arithmetic:** $+$, $-$, $*$, $/$ (with protection for division by zero).
 - **Logical:** if-else.



Example: GP Hyperheuristics for BPP

Sample Heuristic

A randomly generated heuristic might look like:

IF RemainingBinSpace – ItemSize $>$ 0.2 THEN place item ELSE open a new bin.

Fitness Evaluation

1. Apply the heuristic to pack the items.
2. Count the number of bins used.
3. Repeat on multiple test cases and average the results.



Example: GP Hyperheuristics for BPP

Genetic Operations

1. Crossover:

- Parent 1: $\text{RemainingBinSpace} - \text{ItemSize}$.
- Parent 2: $\text{ItemSize} + 0.1$.
- Offspring: $(\text{RemainingBinSpace} - \text{ItemSize}) + 0.1$.

2. Mutation:

- Replace a subtree with a random operation, e.g., $\text{RemainingBinSpace} \rightarrow \text{RemainingBinSpace}/2$.



Example: GP Hyperheuristics for BPP

Result

After evolving heuristics over multiple generations, the best heuristic might be:

IF RemainingBinSpace > 0.5 THEN place item ELSE open a new bin.

This heuristic can generalize well across various instances of the BPP.



Summary

- **Genetic programming hyper-heuristic** can be used to build heuristics or heuristic components.
 - Often, they operate in a *train and test* fashion and
 - Training on selected sample instances could take long time while application to unseen instances is generally fast.
 - Each tree generated by GP can be evaluated using an indicator (evaluation function) showing how good it is in building high quality solutions to the sample problem instances, such as, mean quality of solutions over the sample instances.

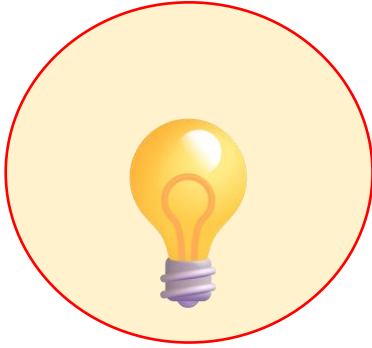


Summary

- A search method (heuristics) with different components, algorithmic configurations and/or parameter settings often performs differently.
- A **metaheuristic** performs search over space of solutions while a hyper-heuristic (which can be a metaheuristic) performs search over the space of (low level) heuristics.
- **Selection hyper-heuristics** can be used to mix perturbative as well as constructive low-level heuristics (**generation hyper-heuristics**).
- The choice of low-level heuristics used in a hyper-heuristic approach influences its **performance**.



Summary



1. Designing Metaheuristics for Examination Timetabling
2. Graph-based Hyper-heuristic
3. Hyper-heuristic Framework
4. Generation Hyper-heuristics
5. Genetic Programming



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



University of
Nottingham

UK | CHINA | MALAYSIA

Questions



University of
Nottingham

UK | CHINA | MALAYSIA

NEXT:

**Advanced Search
Techniques and
Applications**